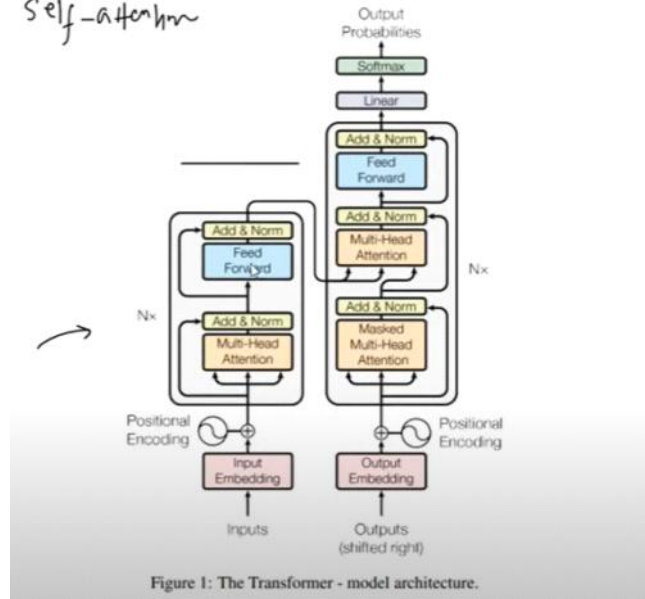


Transformer (Self-Attention is all you need)

30 May 2024 07:43

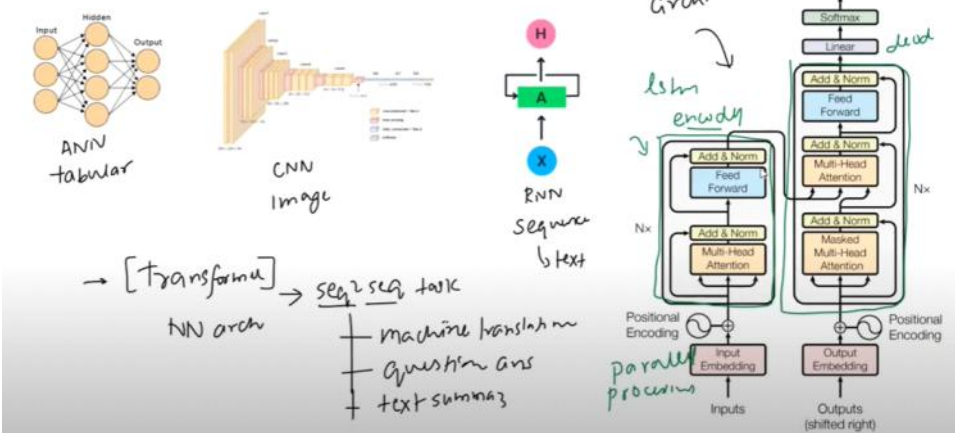


Self-attention

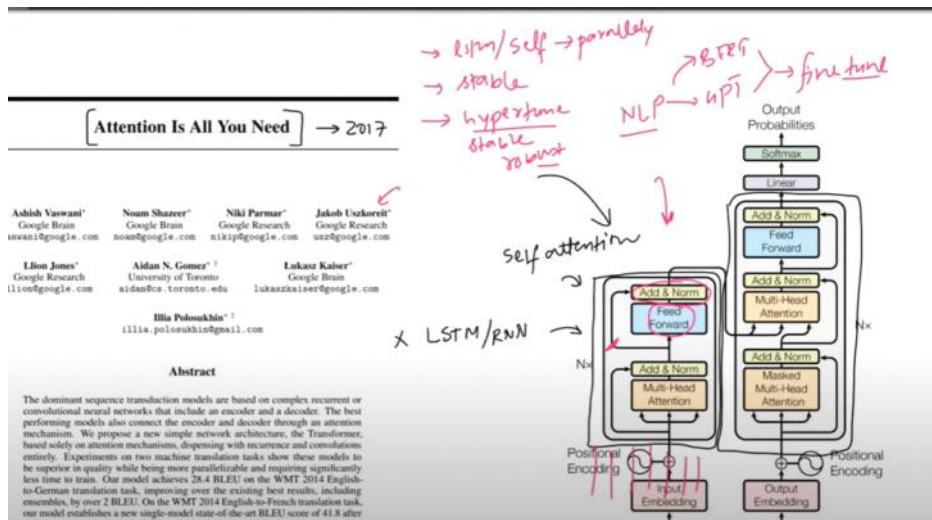
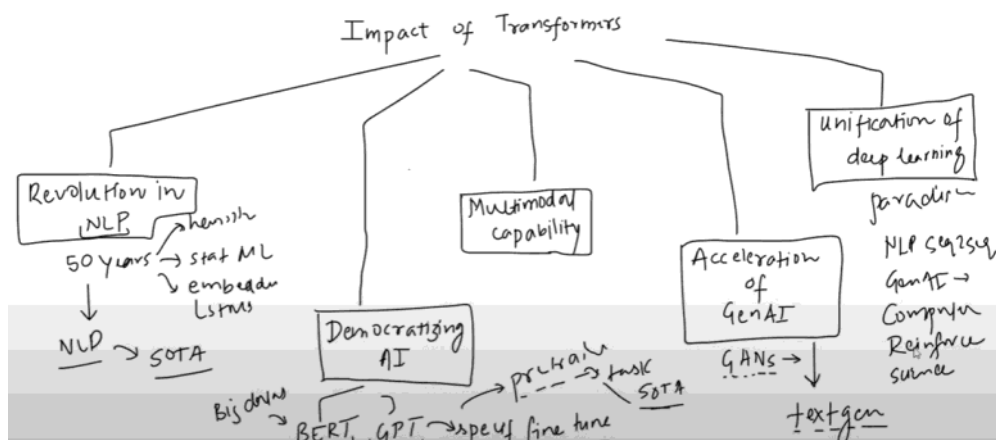


What is Transformer?

27 January 2024 18:43



Transformers are Neural Network architecture built for Seq2Seq Task like machine translation, question answer, text summarization. It has an encoder and decoder unit and we do not have LSTM but have a self attention unit which has the capability to translate, summarize large paragraph at one go. Hence highly scalable, as we are doing parallel processing



LSTM were replaced with Self-Attention, Parallel processing can be done and can be trained in huge datasets and it can be used as base model for transfer learning

The Timeline

28 January 2024 00:55

2000 - 2014 → RNNs / LSTMs

2014 → Attention

2017 → Transformer

2018 → BERT / GPT (Transfer learning)

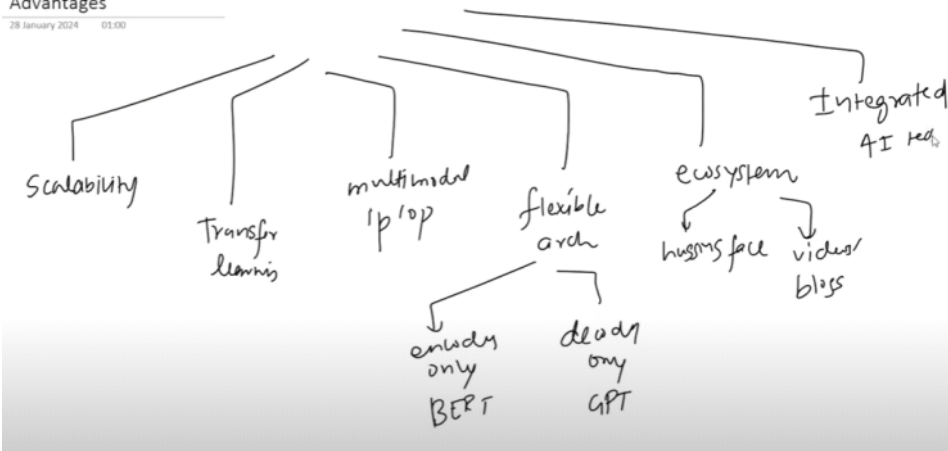
2018 - 2020 → Vision Transf / AlphaFold-2

2021 → Gen AI

2022 → ChatGPT / Stable Diffusion

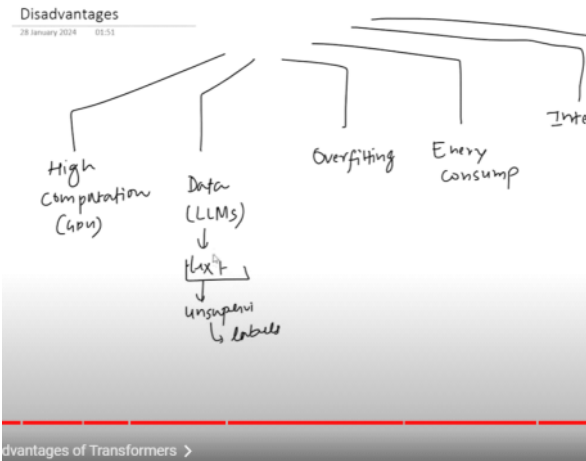
Advantages

28 January 2024 01:00



Disadvantages

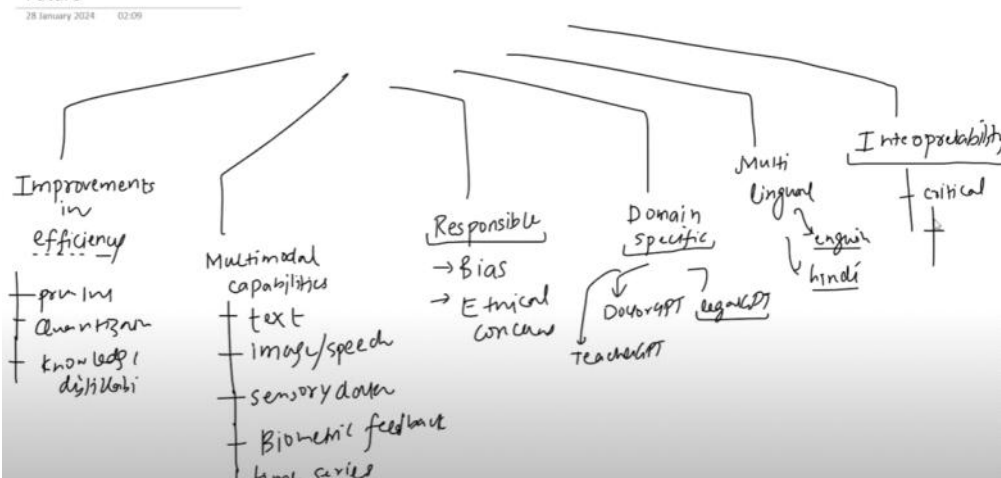
28 January 2024 01:11



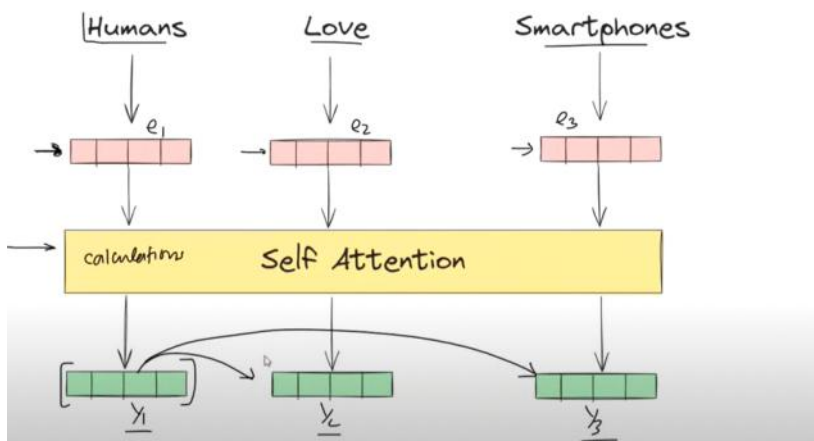
Advantages of Transformers >

Future

28 January 2024 02:09



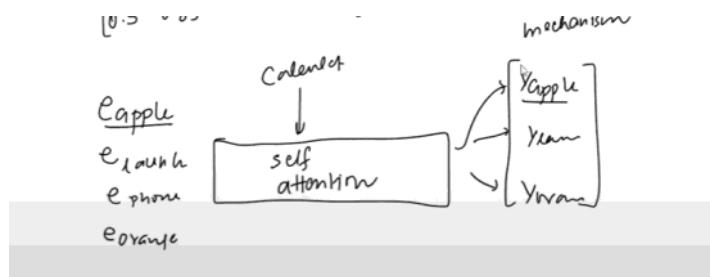
Self attention



Self attention : it provides smart contextual embedding compared to traditional static word embedding as present in Word2Vec, Glove (these are static embedding, i.e. once created after training will be used in subsequent sentences)

It is a mechanism that will generate smart contextual embeddings from word embedding

We send each and every word embedding and then some calculation happens which will generate new smart contextual embedding for each words



S1

$$\begin{aligned} \text{money} &= 0.7 \text{ money} + 0.2 \text{ bank} + 0.1 \text{ grows} \\ \text{bank} &= 0.25 \text{ money} + 0.7 \text{ bank} + 0.05 \text{ grows} \\ \text{grows} &= 0.1 \text{ money} + 0.2 \text{ bank} + 0.7 \text{ grows} \end{aligned}$$

S2

$$\begin{aligned} \text{river} &= 0.8 \text{ river} + 0.15 \text{ bank} + 0.05 \text{ flows} \\ \text{bank} &= 0.2 \text{ river} + 0.78 \text{ bank} + 0.02 \text{ flows} \\ \text{flows} &= 0.4 \text{ river} + 0.01 \text{ bank} + 0.59 \text{ flows} \end{aligned}$$

n dim

$$\begin{aligned} e_{\text{money}}^{(\text{new})} &= 0.7 e_{\text{money}} + 0.2 e_{\text{bank}} + 0.1 e_{\text{grows}} \\ e_{\text{bank}}^{(\text{new})} &= 0.25 e_{\text{money}} + 0.7 e_{\text{bank}} + 0.05 e_{\text{grows}} \\ e_{\text{grows}}^{(\text{new})} &= 0.1 e_{\text{money}} + 0.2 e_{\text{bank}} + 0.7 e_{\text{grows}} \end{aligned}$$

0.7 means that E_{money} (new contextual embedding of money) is 0.7 times the previous embedding of the Money and so on. (i.e. 0.2 times the previous embedding of bank and so on. So these numbers are representing similarity between each word embedding)

Embeddings are vectors in high dimensional spaces. To find the **similarity** between 2 vectors we do a dot product between vectors. Higher the dot product value between 2 vectors more closer the vectors are to each other

bank = 0.25 money + 0.7 bank + 0.05 grows

grows = 0.1 money + 0.2 bank + 0.7 grows

flows = 0.4 river + 0.01 bank + 0.59 flows

n dim

$$\begin{aligned} e_{\text{money}}^{(\text{new})} &= 0.7 e_{\text{money}} + 0.2 e_{\text{bank}} + 0.1 e_{\text{grows}} \\ e_{\text{bank}}^{(\text{new})} &= 0.25 e_{\text{money}} + 0.7 e_{\text{bank}} + 0.05 e_{\text{grows}} \\ e_{\text{grows}}^{(\text{new})} &= 0.1 e_{\text{money}} + 0.2 e_{\text{bank}} + 0.7 e_{\text{grows}} \end{aligned}$$

similarity

dot products

24 + 2 = 26

6 + 5 = 11

S1

$$\begin{aligned} \text{money} &= 0.7 \text{ money} + 0.2 \text{ bank} + 0.1 \text{ grows} \\ \text{bank} &= 0.25 \text{ money} + 0.7 \text{ bank} + 0.05 \text{ grows} \\ \text{grows} &= 0.1 \text{ money} + 0.2 \text{ bank} + 0.7 \text{ grows} \end{aligned}$$

S2

$$\begin{aligned} \text{river} &= 0.8 \text{ river} + 0.15 \text{ bank} + 0.05 \text{ flows} \\ \text{bank} &= 0.2 \text{ river} + 0.78 \text{ bank} + 0.02 \text{ flows} \\ \text{flows} &= 0.4 \text{ river} + 0.01 \text{ bank} + 0.59 \text{ flows} \end{aligned}$$

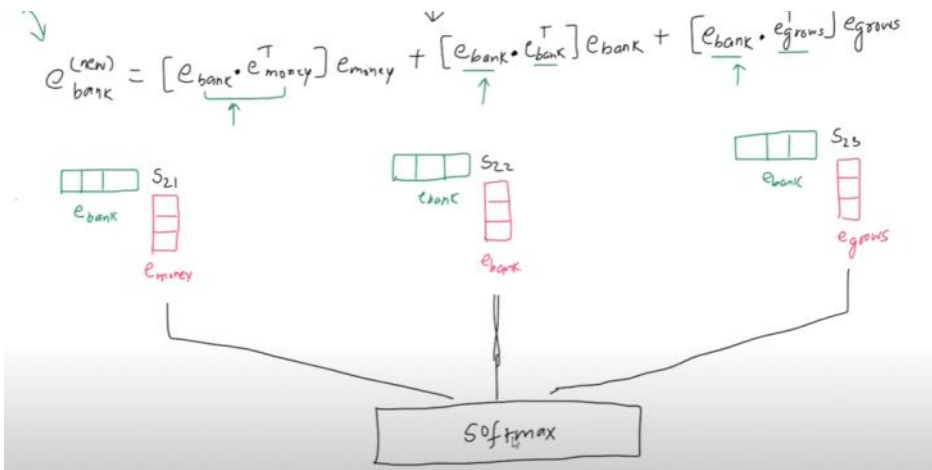
n dim

$$e_{\text{bank}}^{(\text{new})} = [e_{\text{bank}} \cdot e_{\text{money}}^T] e_{\text{money}} + [e_{\text{bank}} \cdot e_{\text{bank}}^T] e_{\text{bank}} + [e_{\text{bank}} \cdot e_{\text{grows}}^T] e_{\text{grows}}$$

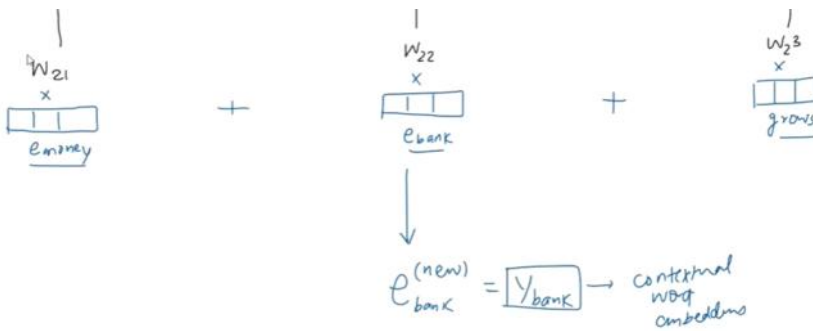
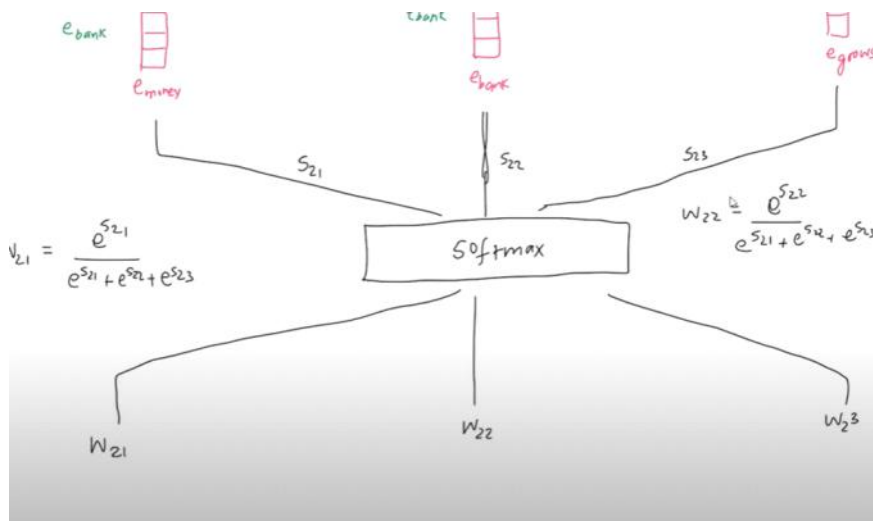
S21

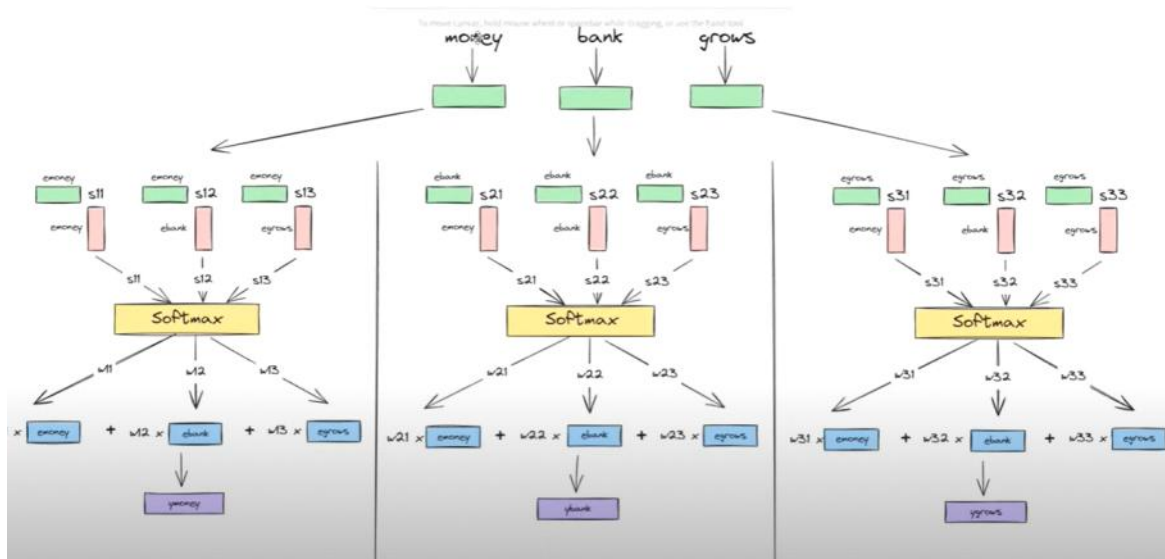
S22

S23



S21, S22, S23 are the dot product of embeddings and there are normalized value using the softmax.





Similarly done for other words as above, but it has a drawback that we will lose the sequence information, i.e. which word has come first and which word has come last. Since the contextual embedding is GENERAL contextual Embedding and NOT task specific contextual embedding.

e.g. in General contextual embedding : A piece of cake - translation will take a literal meaning , while Task specific contextual meaning would have translated this one "work done easily"

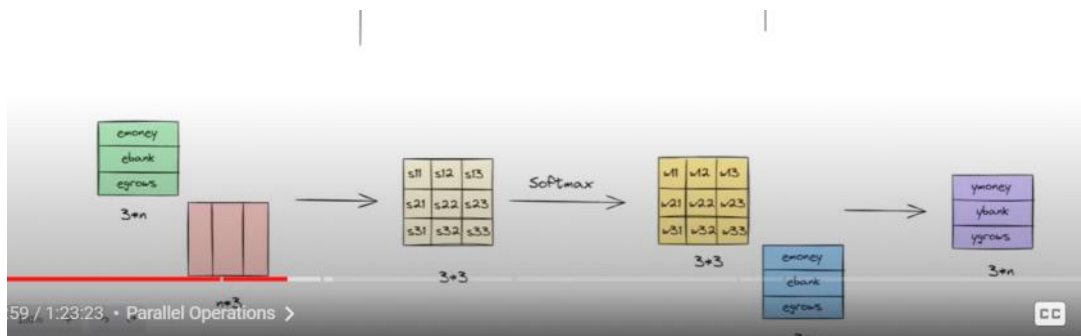
So we need to get task specific contextual embedding, e.g. if we are doing machine translation then for this type of task contextual embedding should be done.

If we are doing sentiment analysis - for this type of task contextual embedding should be done.

This can be done by introducing some learning parameters (i.e. weights and bias so that the task specific contextual embedding can be done)

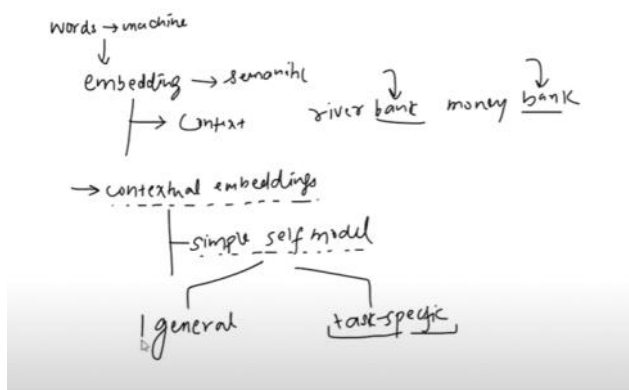
Parallelize the embedding calculation as below, another advantage is that there is NO learning parameters.

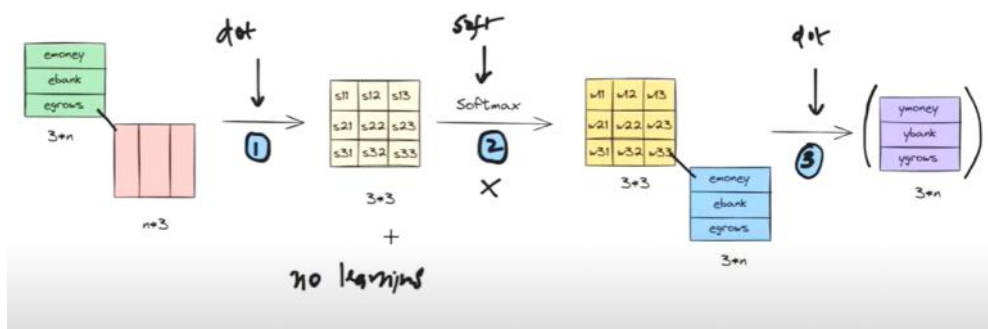
3 = no. of words
n = Embedding vector size



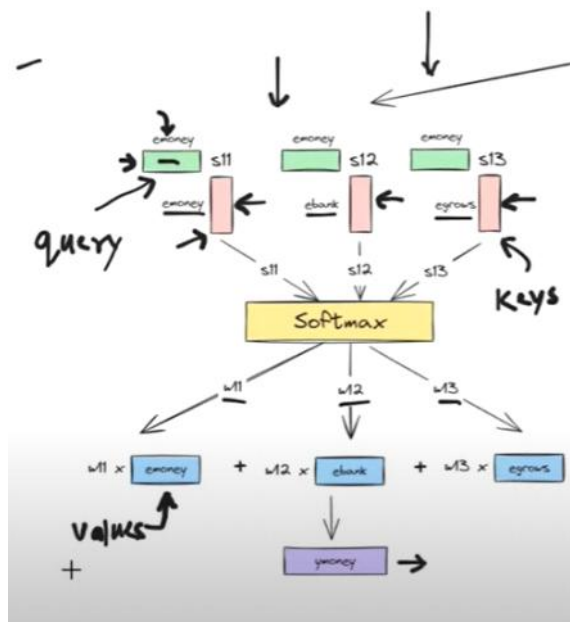
Progress

06 February 2024 00:42



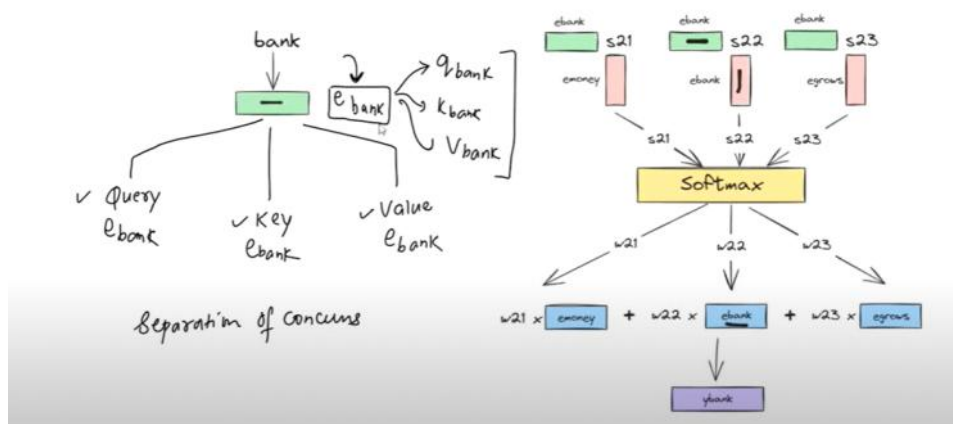


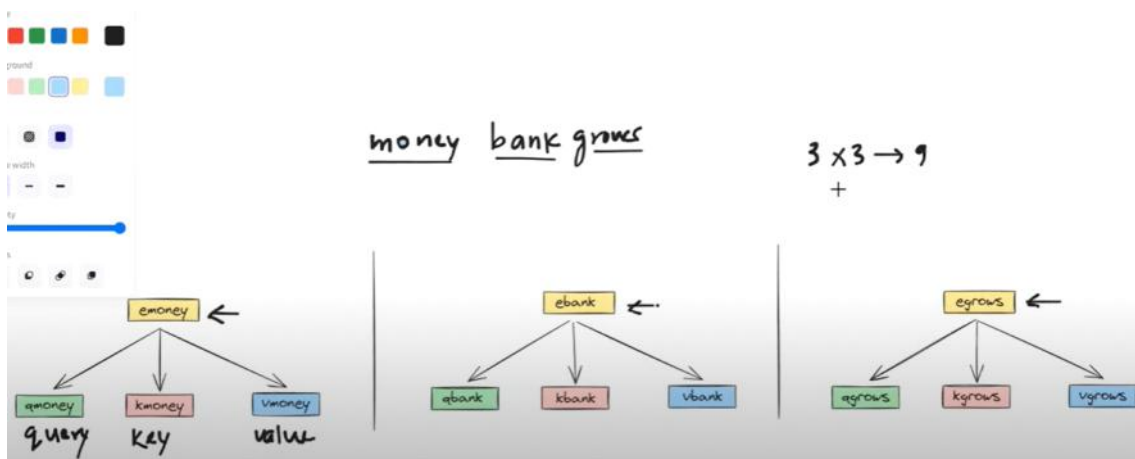
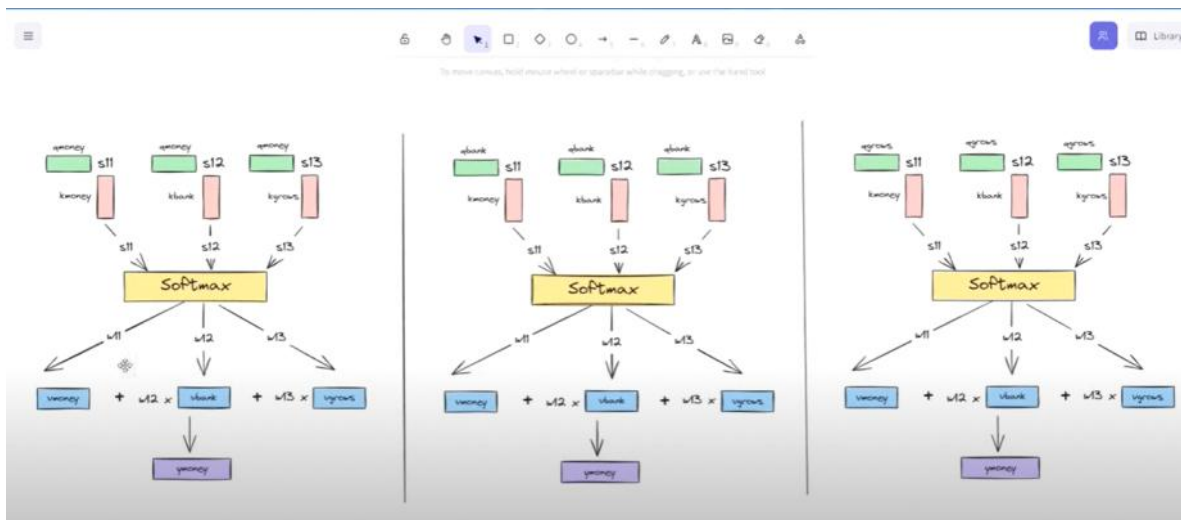
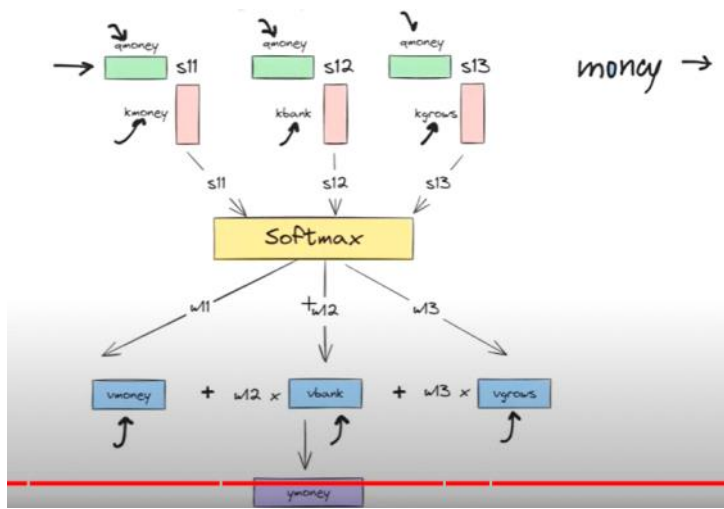
Weights / Biases (learnable parameters) can be introduced at step 1 and 3



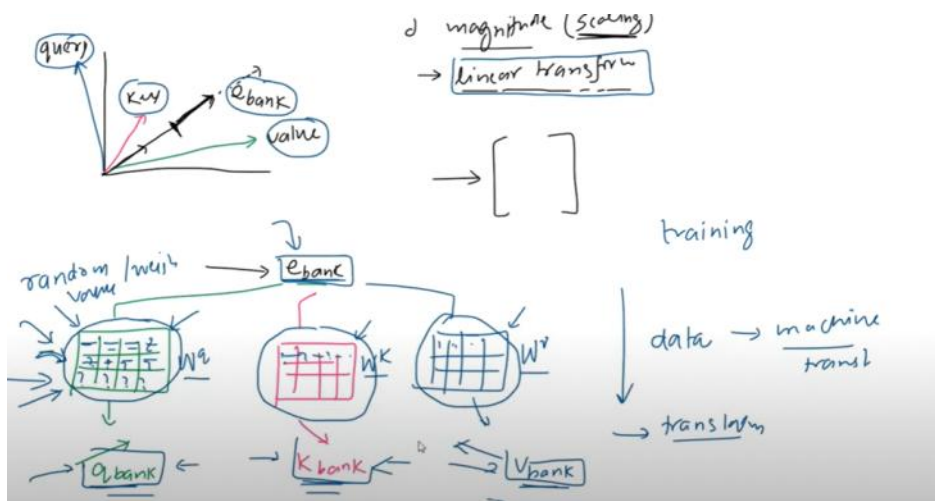
Query, Key & Value Vectors

06 February 2024 13:32

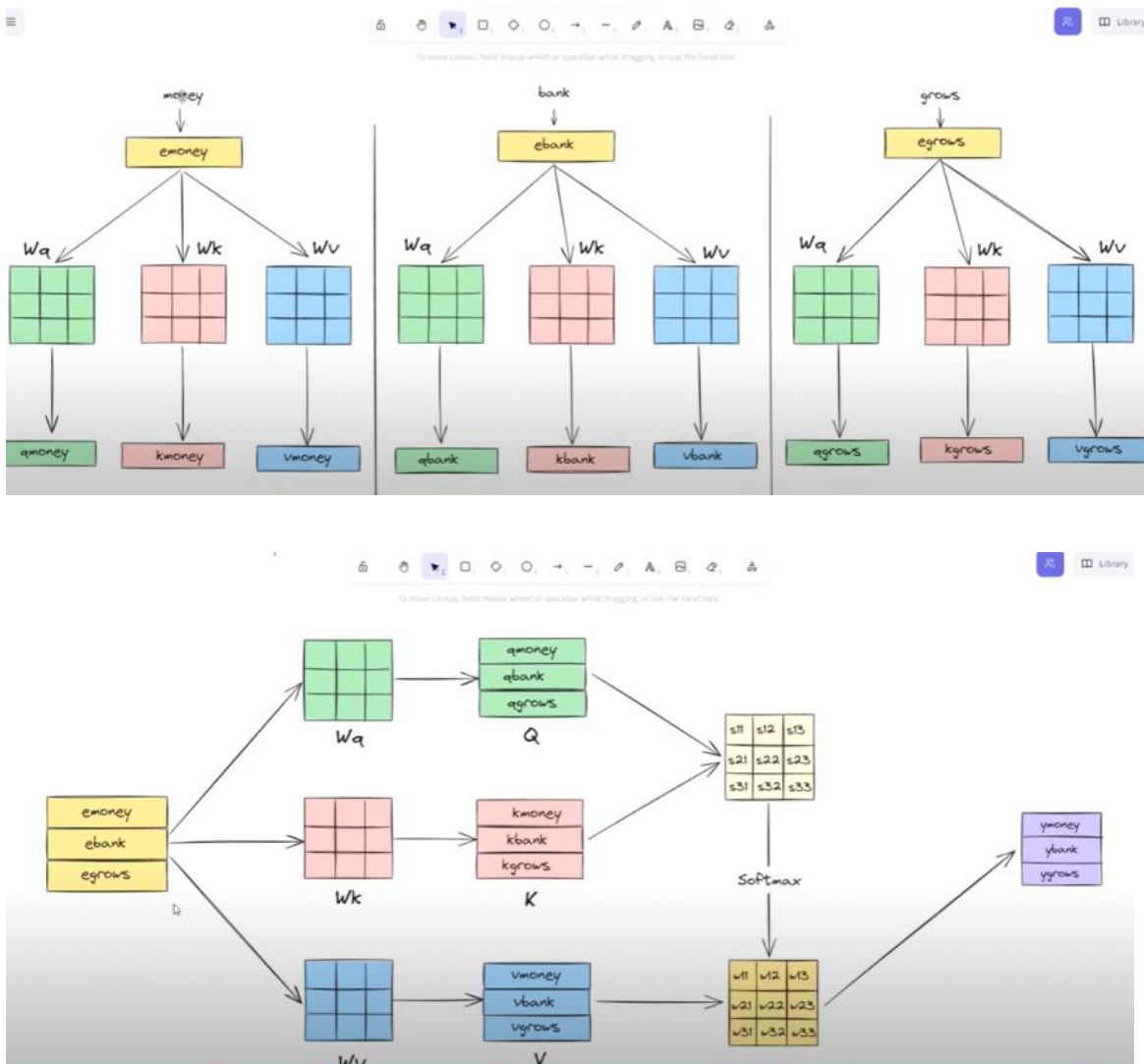


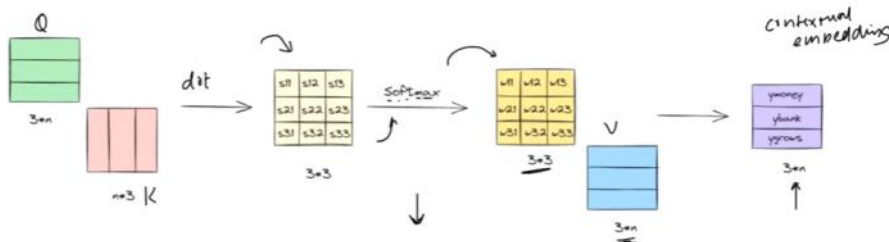


From Embedding vector we need to create 3 different vectors (Query, Key and Value Vector). This will be done by multiplying the embedding vector with some vector (i.e. linear transformation)

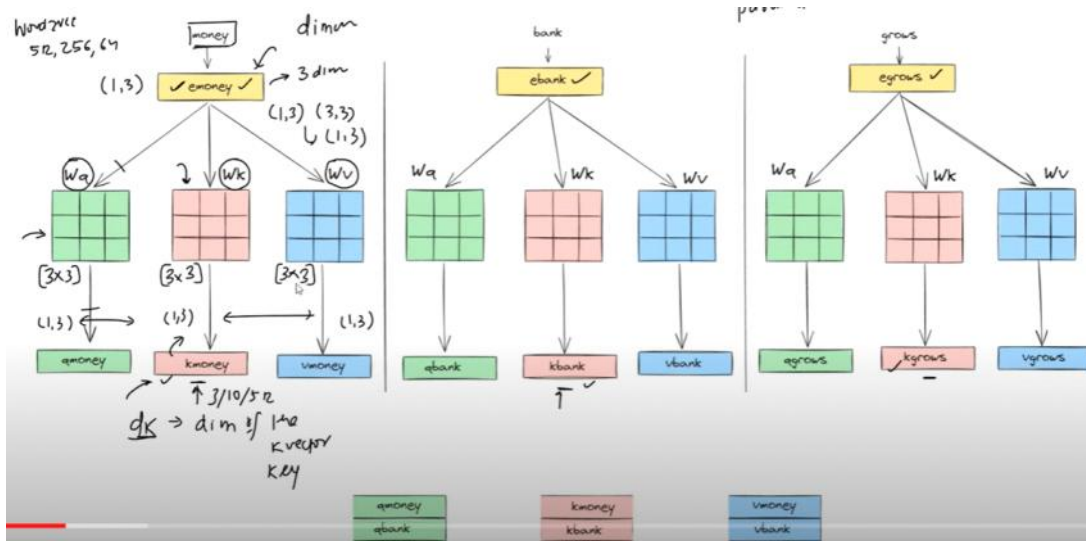


For linear transformation, the matrix used for generating query, key and Value vectors are random weight vector(initialized) and then during training the weights of the matrix will get updated during backpropagation





$$\text{Attention}(Q, K, V) = \text{Softmax}(Q K^T) V$$



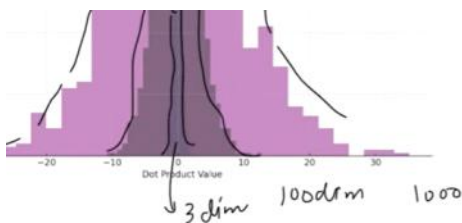
$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

why?

$\sqrt{d_k}$

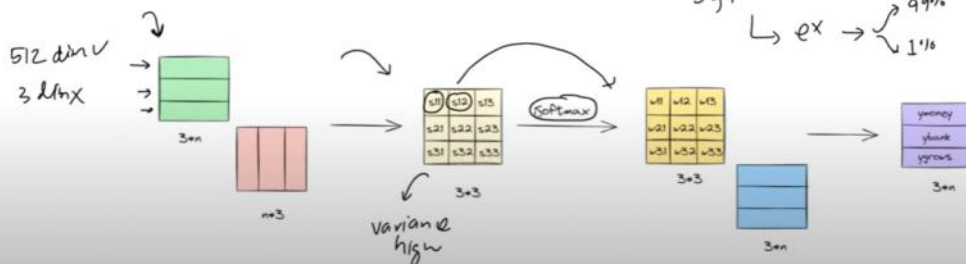
Dot-product is Natural

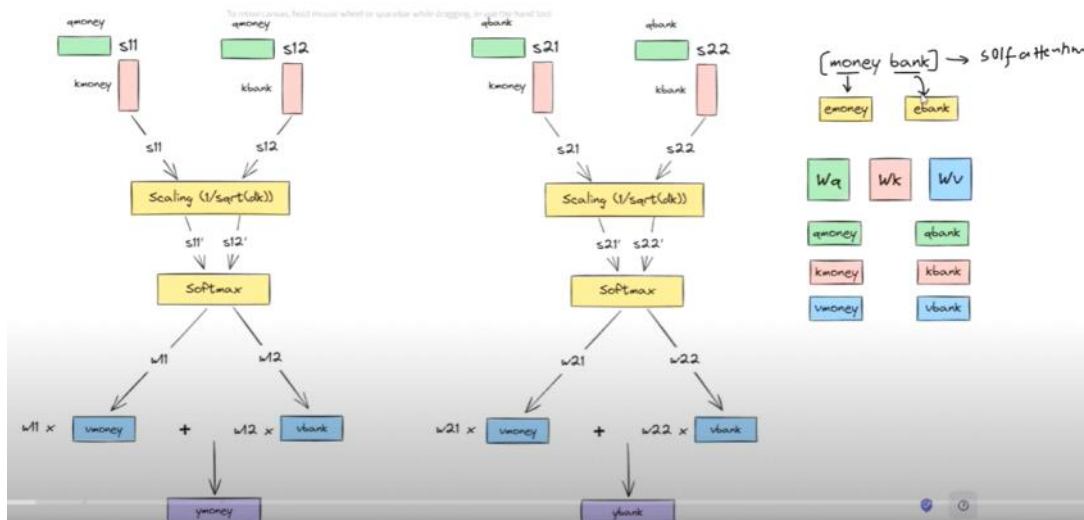
low dimensional $\rightarrow$ dot product $\rightarrow$ low variance
high dim $\rightarrow$ dot product $\rightarrow$ high variance



high variance
 $\downarrow$
it's a problem

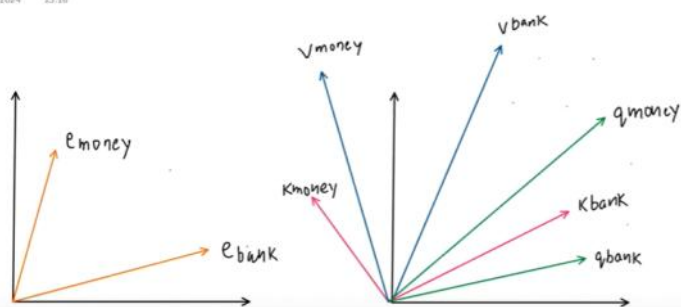
softmax $\rightarrow$ prob $\rightarrow$ 100/12
 $\rightarrow$ ex $\rightarrow$ 99%
 $\rightarrow$ 1%





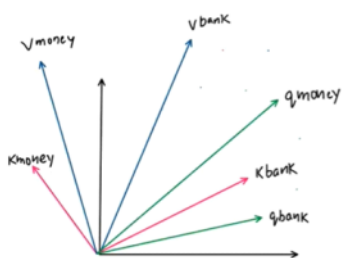
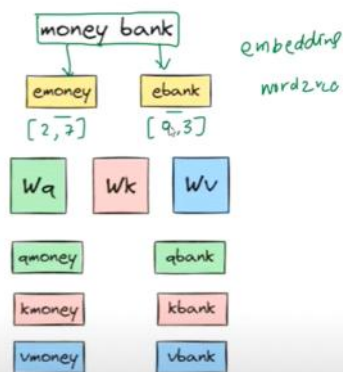
Geometric Intuition

08 March 2024 15:16



$$W_q = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix} \quad W_k = \begin{bmatrix} 3 & 4 \\ 5 & 1 \end{bmatrix} \quad W_v = \begin{bmatrix} 4 & 1 \\ 2 & 1 \end{bmatrix}$$

All values are hypothetical



[Dot Product]

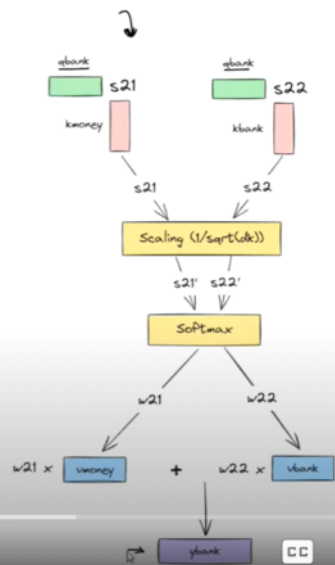
$$s_{21} = 10 \quad s_{22} = 32$$

[Scaling]

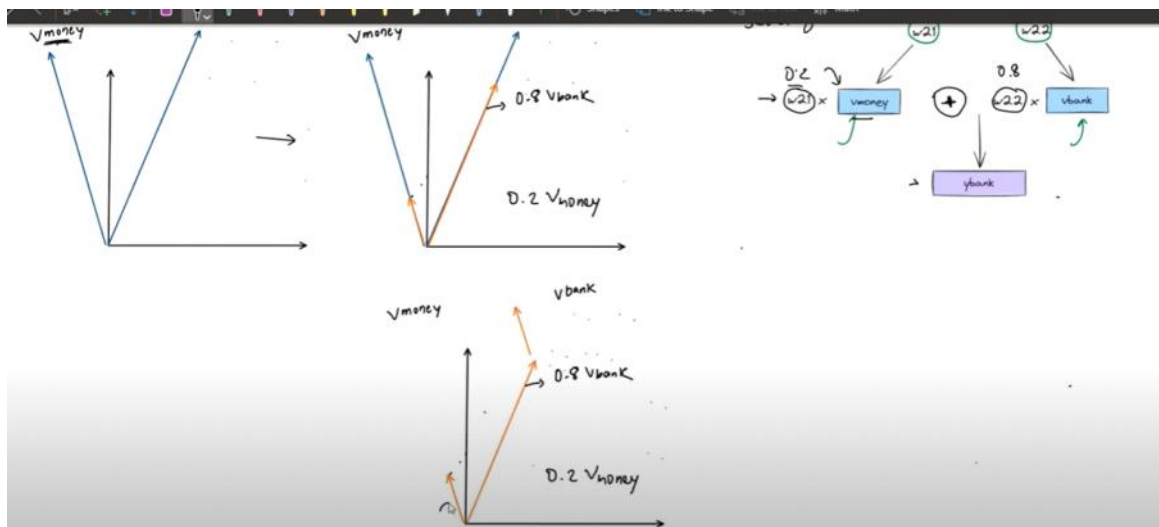
$$s'_{21} = \frac{10}{\sqrt{2}} = 7.07 \quad s'_{22} = \frac{32}{\sqrt{2}} = 22.63$$

[Softmax]

$$w_{21} = 0.2 \quad w_{22} = 0.8$$

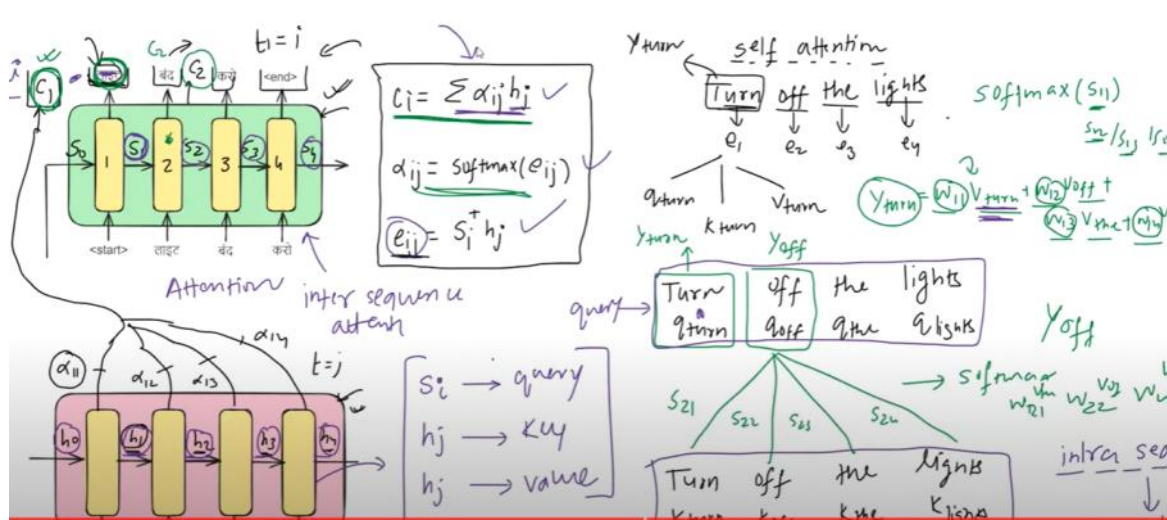
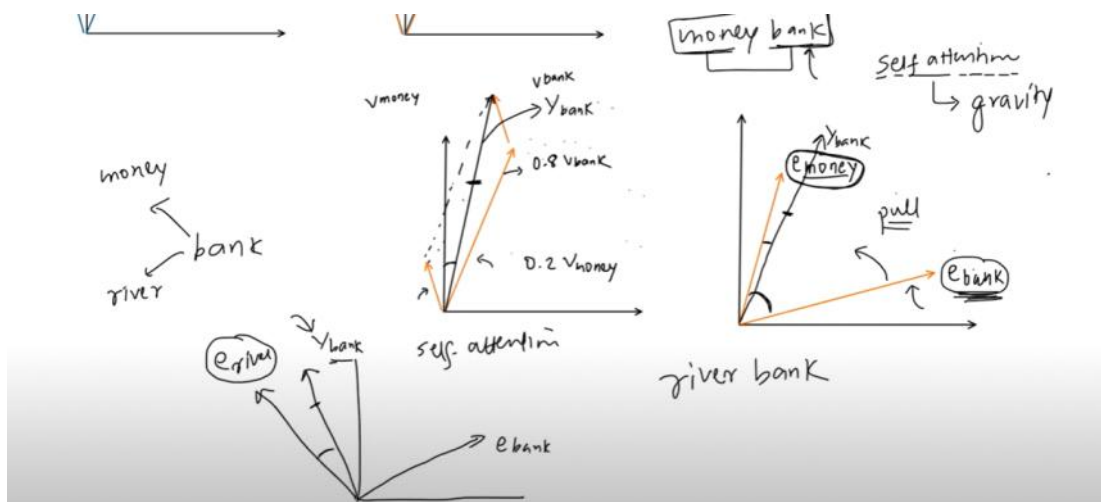


If the angular distance b/w 2 vectors are more the dot product b/w those 2 vectors will be less and vice versa. More the angle b/w the 2 vectors lesser the semantic meaning similarity b/w those 2 vectors



$W_{21} * v_{money}$ = we are doing scaling multiplication. Hence it is scaling operation of the vectors

Addition of vector is done either by parallelogram or triangle law, here we are following parallelogram law, and in this example y_{bank} will get generated more closer to e_{money} .



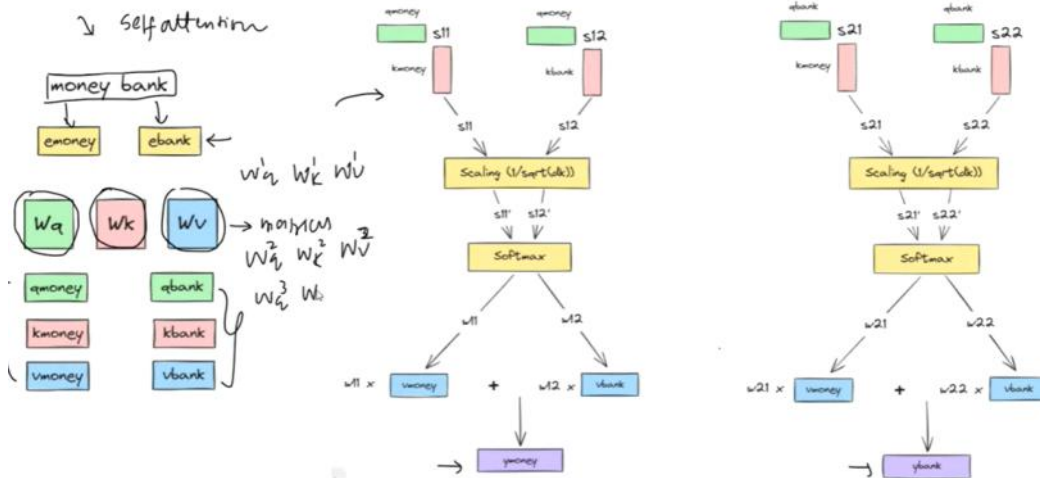
It is called Self in Self attention because it is a intra-sequence (i.e. similarity score is calculated between the same sequence, both same English Sequence (Turn off the Lights) while in Luang attention, the attention is intersequence attention as we are calculating the C_i (similarity score) between 2 sequence (English and Hindi)

Similarity b/w Luang and Self attention

S_i = Qembedding
 h_j = KeyEmbedding
 h_j = ValueEmbedding

he man saw the astronomer with a telescope

self attention →

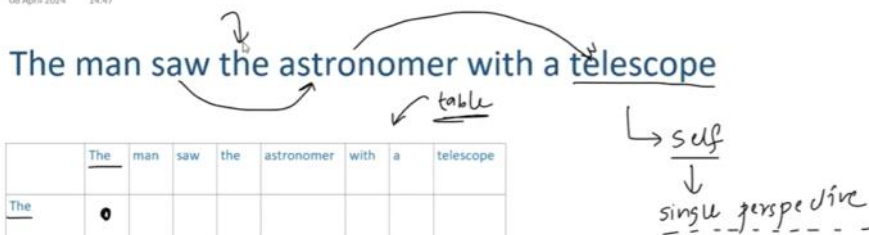


Drawback of Self attention:

1. It generate a contextual embedding and sees a single document with single perspective. So it becomes difficult for ambiguous statement like " The man saw the astronomer with a telescope"
2. IT generates a similarity score b/w words like below:

Problem with Self Attention

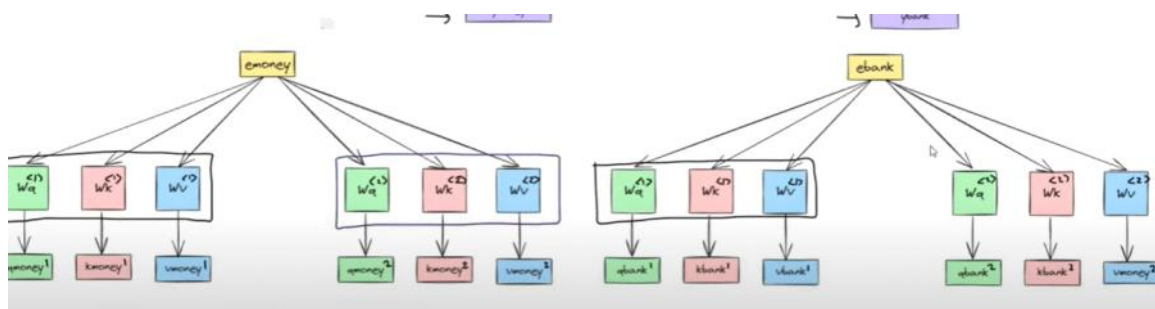
08 April 2024 14:47

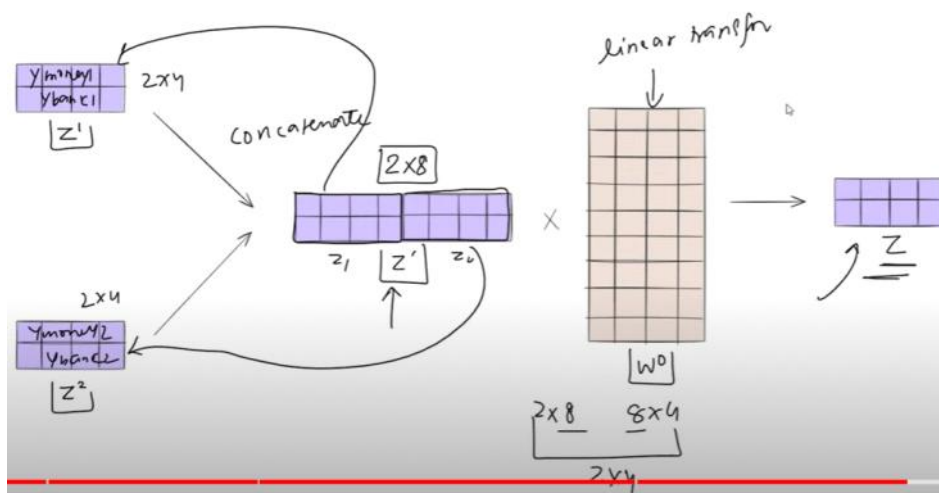
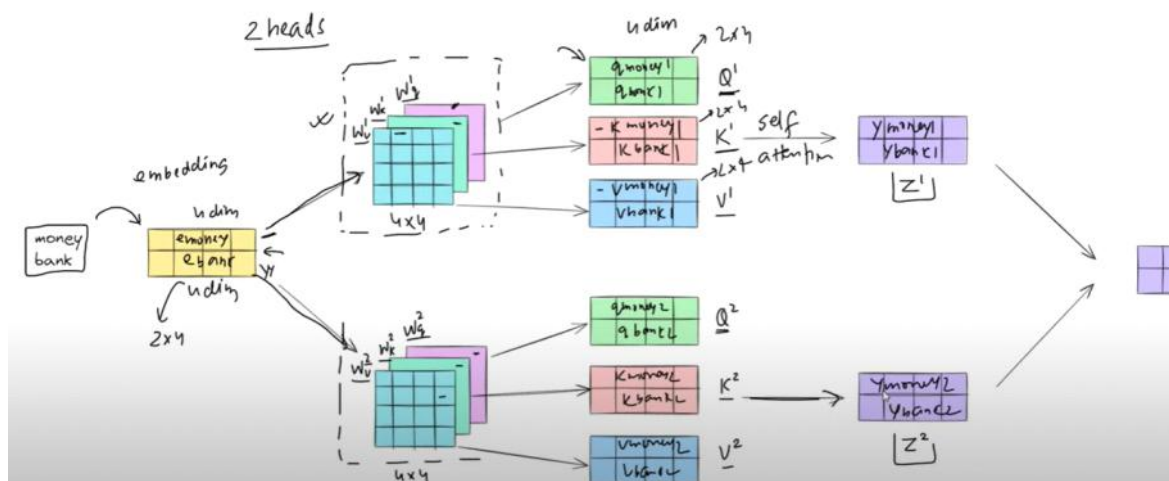
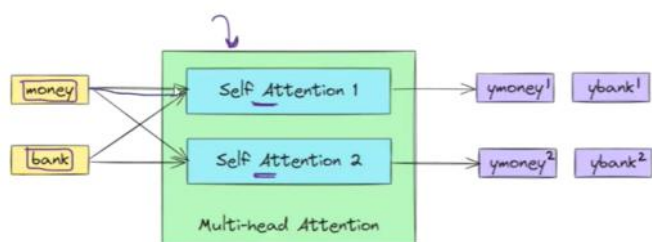
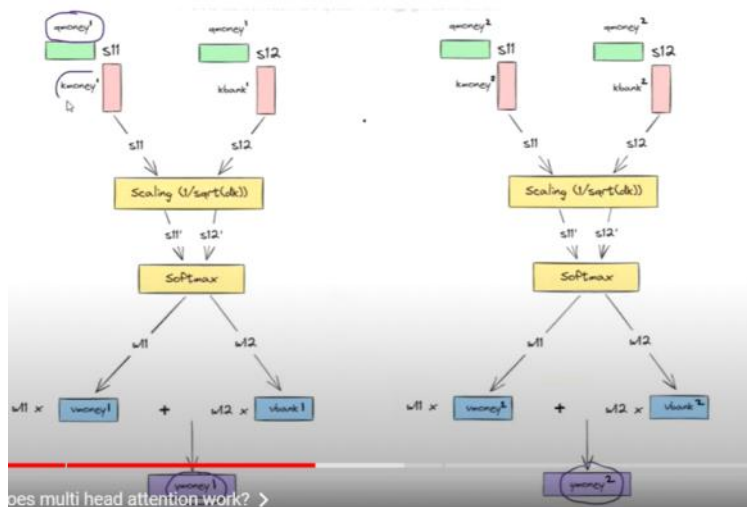


In order to solve the issue, multihead attention came into picture.

Multihead attention = 2 self attention, we increases the matrixes i.e. W_q , W_k , W_v . Example we will have 2 matrixes of W_q , W_k , W_v

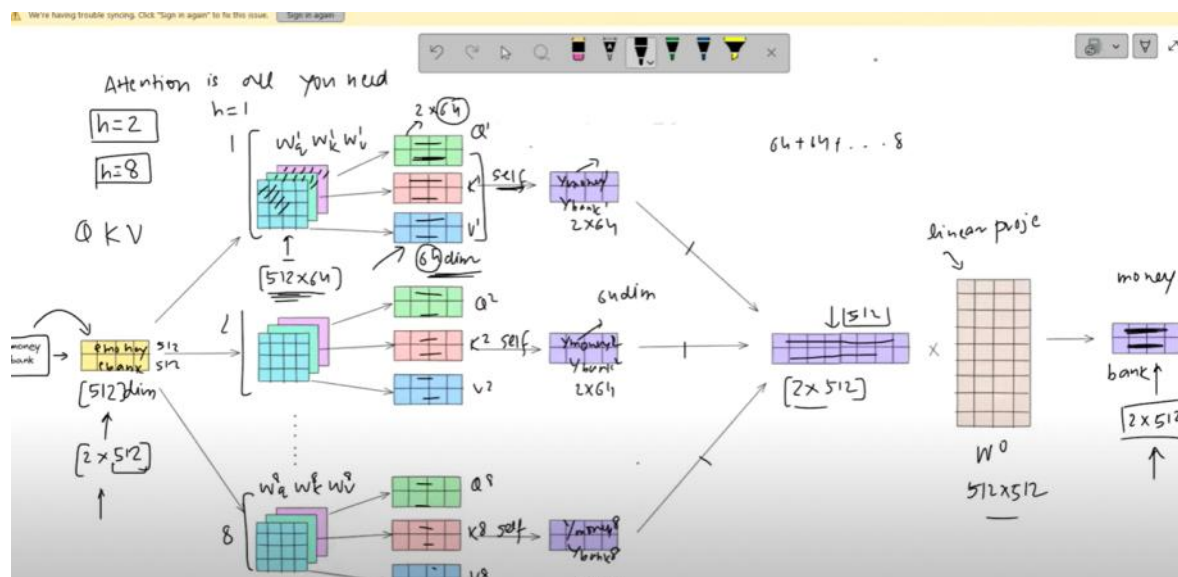
In the paper there are 8 Multihead attentions have been used





The output of the attention heads are concatenated. Since the size becomes 2x8. Since the output has

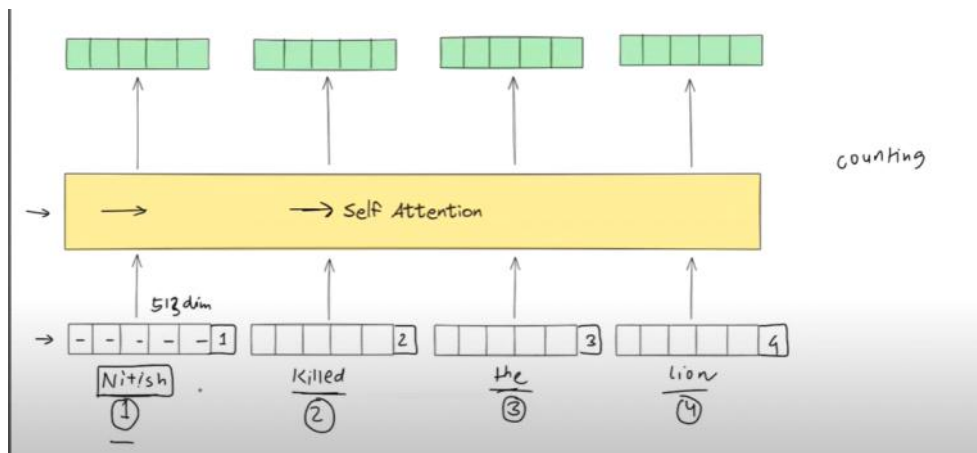
to be in the same shape as the embedding matrix (which was 2×4). Hence we will do a linear transformation to get the output matrix size as 2×4



Positional Encoding:

To send the order of words:

1. Counting



Problem 1) unbounded $\rightarrow$ upper limit (huge)

-1 to 1

NN architecture

big number $\leftarrow$ backprop

Smooth handling, continuous

Problem 2

Discrete
good neural
networks

gradient flow

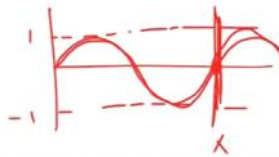
good neural
numerical
stability
counting

Problem 3 → can't relative positionings

Nitish killed the lion
 [1] [2] [3] [4]
 absolute position

Problem
 → unbounded (bounded) ✓
 → discrete (continuous) ✓
 → relative (periodic) ✓
 function →

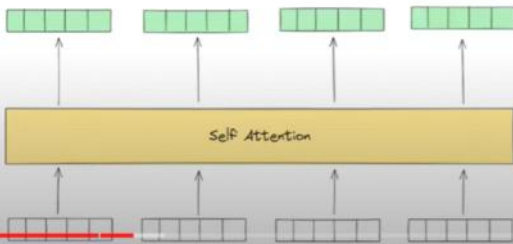
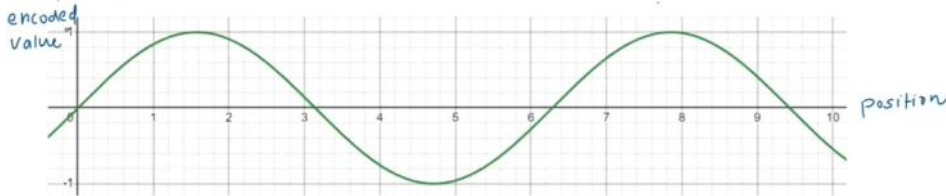
trigonometric
 ↓
 sine



positional encoder → sine
 better so

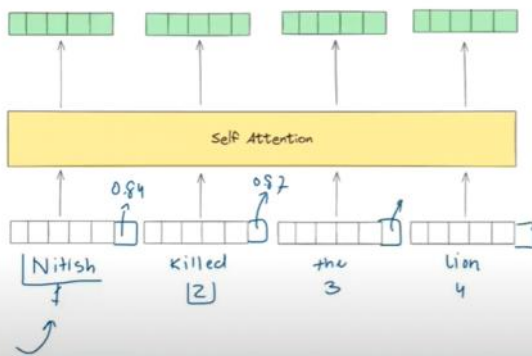
The sine function as a solution

23 May 2024 17:17



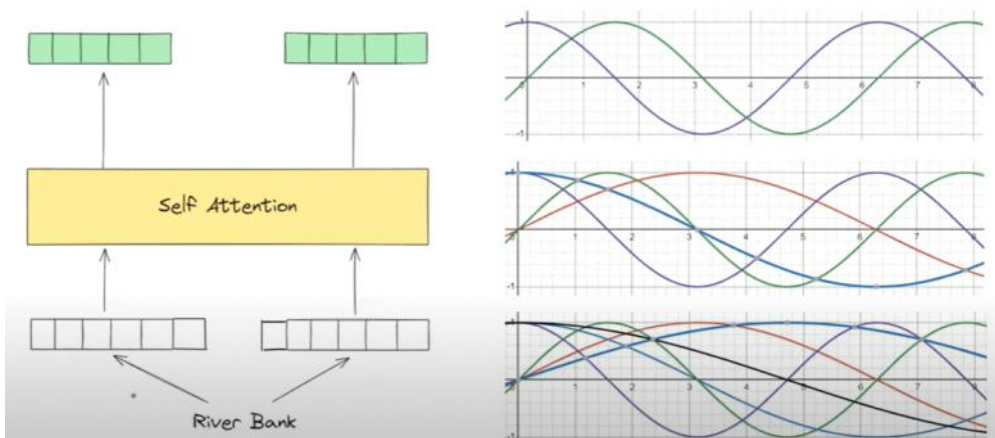
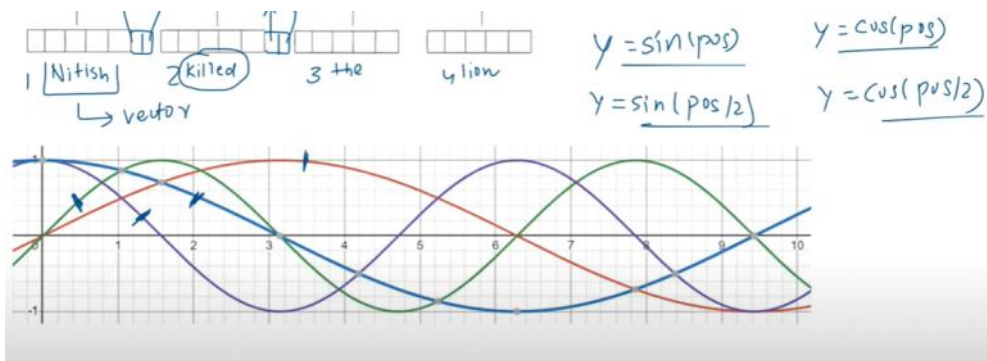
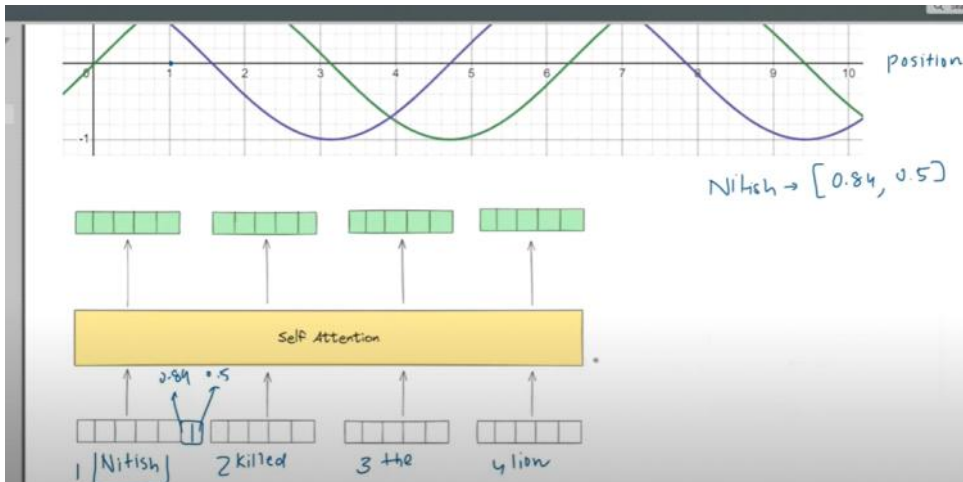
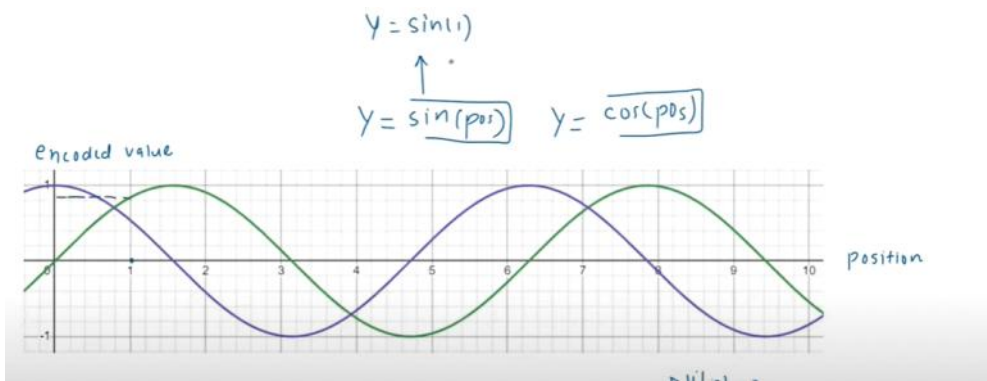
encoded value
 $y = \sin(\text{position})$
 $y = \sin(1) = 0.84$
 $y = \sin(2)$

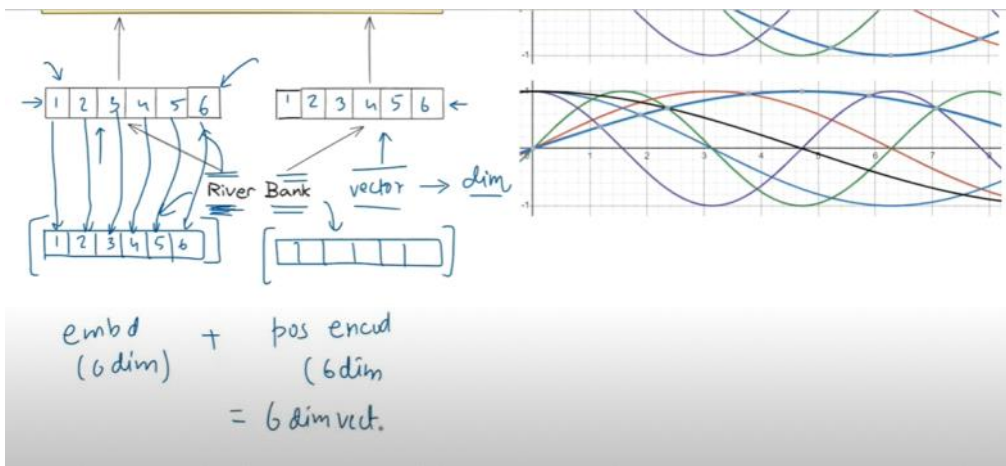
4 • The sine function as a solution >



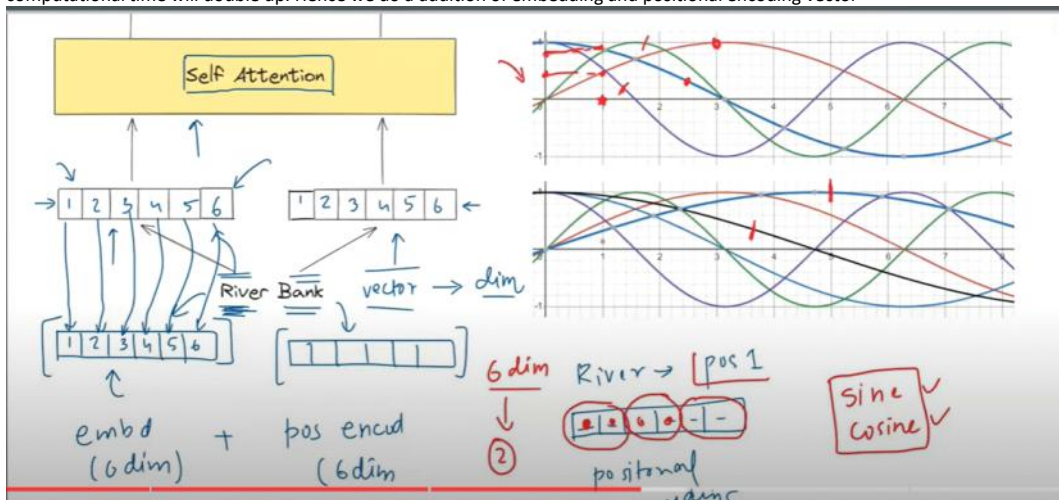
encoded value
 $y = \sin(\text{position})$
 $y = \sin(1) = 0.84$
 $y = \sin(2) = 0.92$
 $y = \sin(3) = 0.14$
 $y = \sin(4) = -0.90$

Drawback of using single $\sin(\text{position})$ was the relative parameter, i.e. 2 different words can have same $\sin(\text{position value})$, since the pattern of sine curve repeats, so for 2 different words present at 2 different positions might give the same value, which is a problem.
 In order to solve the above issue, we take 2 trigonometry function, sine and cos of position





Positional encoding vector dimension will be equal to the dimension size embedding vector
 Addition of embedding and positional encoding which will be a 6 dimension (example) and this new vector is passed to the self-attention block. We do not concatenate the embedding and positional encoding as the dimension will double up and the computational time will double up. Hence we do a addition of embedding and positional encoding vector



pos → position
 pos=0 pos=1
 $d_{model} \rightarrow \dim$ of embedding
 $d_{model}=6$

for $i=0 \rightarrow pos=0$

$$PE(0,0) = \sin\left(\frac{0}{10000^0}\right) = 0$$

$$PE(0,1) = \cos\left(\frac{0}{10000^0}\right) = 1$$

for $i=1$

$$PE(1,2) = \sin\left(\frac{1}{10000^{1/3}}\right) = 0.04$$

$$PE(1,3) = \cos\left(\frac{1}{10000^{1/3}}\right) = 0.99$$

for $i=2$

$$PE(2,4) = \sin\left(\frac{1}{10000^{2/3}}\right) = 0.00$$

$$PE(2,5) = \cos\left(\frac{1}{10000^{2/3}}\right) = 0.99$$

for $i=0$

$$PE(0,2) = \sin\left(\frac{0}{10000^{1/3}}\right) = 0$$

$$PE(0,3) = \cos\left(\frac{0}{10000^{1/3}}\right) = 1$$

for $i=1$

$$PE(1,2) = \sin\left(\frac{1}{10000^{1/3}}\right) = 0.04$$

$$PE(1,3) = \cos\left(\frac{1}{10000^{1/3}}\right) = 0.99$$

for $i=2$

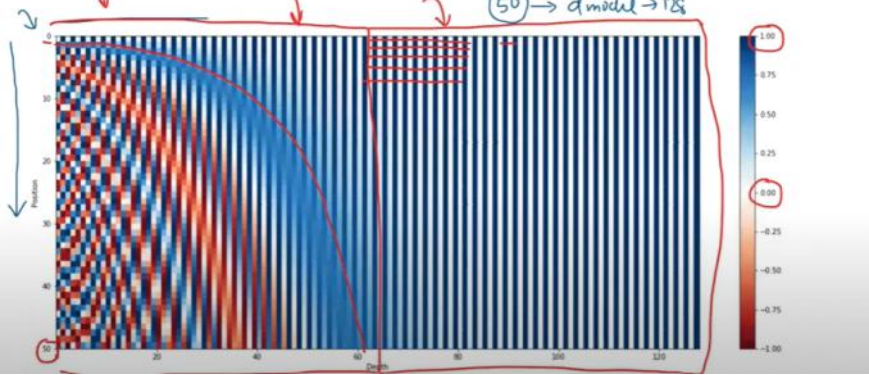
$$PE(2,4) = \sin\left(\frac{1}{10000^{2/3}}\right) = 0.00$$

$$PE(2,5) = \cos\left(\frac{1}{10000^{2/3}}\right) = 0.99$$

Interesting Observations

24 May 2024 02:06

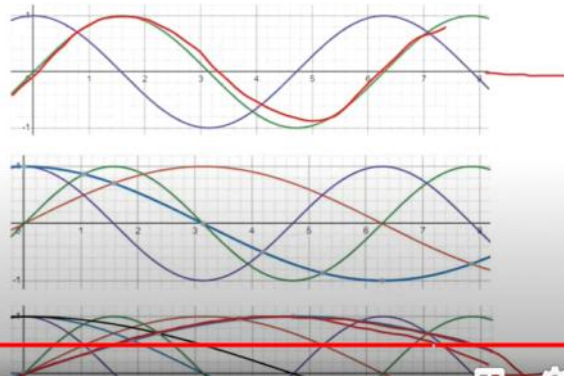
Sentence $\rightarrow$ 50 words
 $\downarrow$ $\rightarrow$ embedding $\rightarrow$ 128 dmod
 (50) $\rightarrow$ dmod $\rightarrow$ 128



For Higher number of words (in this case 50 words) Low frequency function is used

100 binary encoding

0:	0 0 0 0	8:	1 0 0 0
1:	0 0 0 1	9:	1 0 0 1
2:	0 0 1 0	10:	1 0 1 0
3:	0 0 1 1	11:	1 0 1 1
4:	0 1 0 0	12:	1 1 0 0
5:	0 1 0 1	13:	1 1 0 1
6:	0 1 1 0	14:	1 1 1 0
7:	0 1 1 1	15:	1 1 1 1

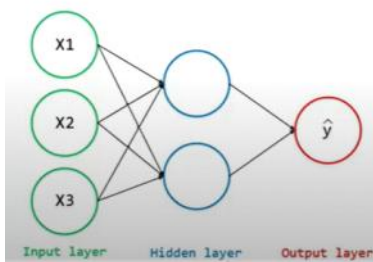


What is Normalization

05 June 2024 10:32

Normalization in deep learning refers to the process of transforming data or model outputs to have specific statistical properties, typically a mean of zero and a variance of one.

What do we normalize?



$$\begin{matrix} f_1 \\ 27 \\ 23 \\ u_1 \end{matrix} \quad \begin{matrix} \text{min-max} \\ \\ \\ \end{matrix}$$

$$\frac{x_i - \mu}{\sigma} \rightarrow \begin{matrix} \mu = 0 \\ \sigma = 1 \end{matrix}$$

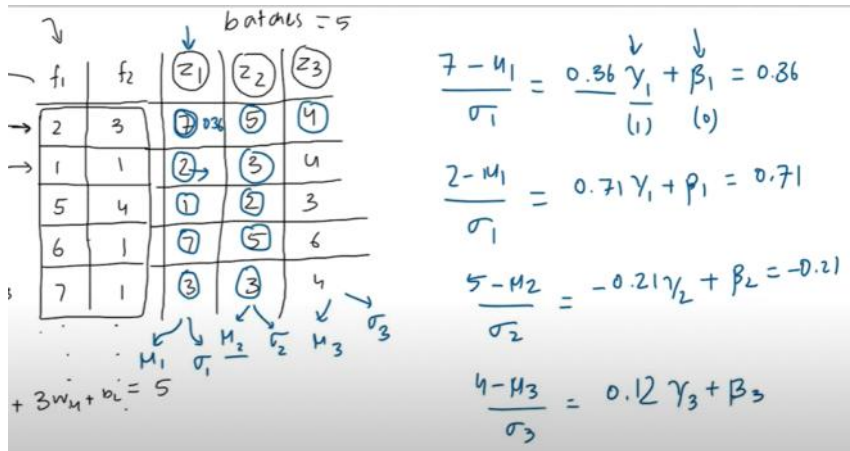
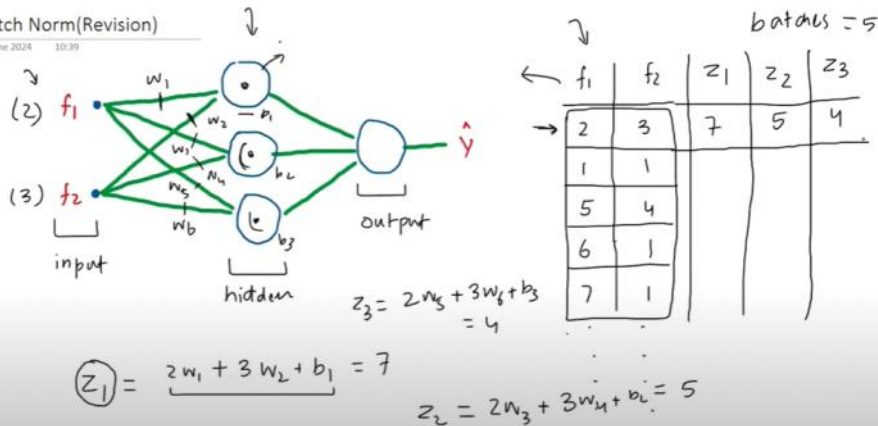
In deep learning, normalization is applied to the input data, so the normalized input data can be passed to the hidden layer. Normalization is also applied to the output of the hidden layer . i.e. on the activations of the hidden layer

Benefits of Normalization in Deep Learning

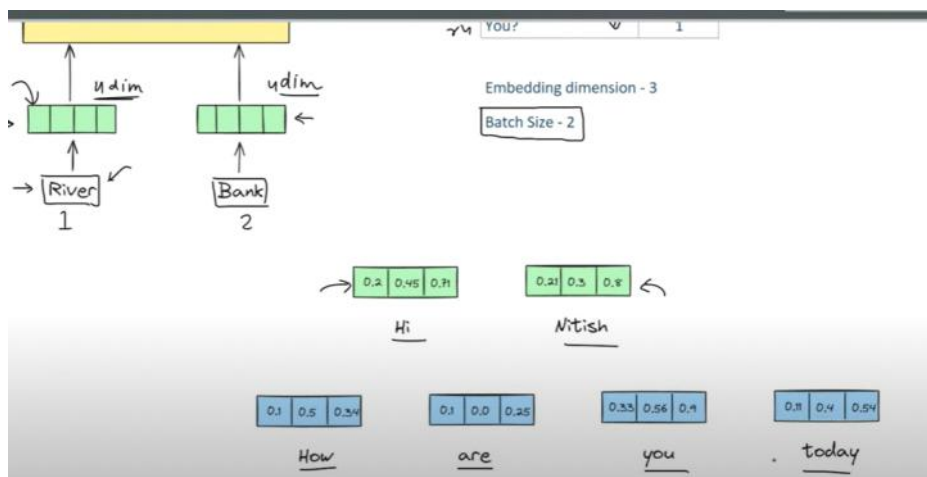
- Improved Training Stability:
 - Normalization helps to stabilize and accelerate the training process by reducing the likelihood of extreme values that can cause gradients to explode or vanish.
- Faster Convergence:
 - By normalizing inputs or activations, models can converge more quickly because the gradients have more consistent magnitudes. This allows for more stable updates during backpropagation.
- Mitigating Internal Covariate Shift:
 - Internal covariate shift refers to the change in the distribution of layer inputs during training. Normalization techniques, like batch normalization, help to reduce this shift, making the training process more robust.
- Regularization Effect:
 - Some normalization techniques, like batch normalization, introduce a slight regularizing effect by adding noise to the mini-batches during training. This can help to reduce overfitting.

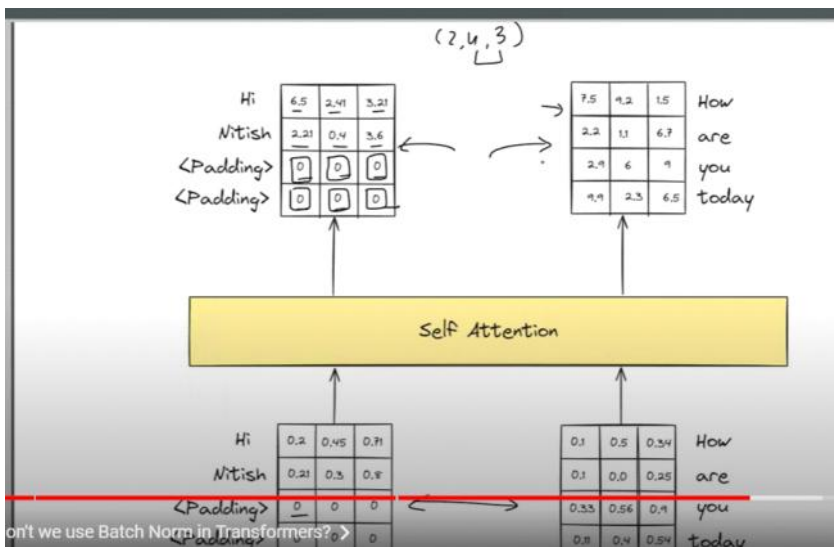
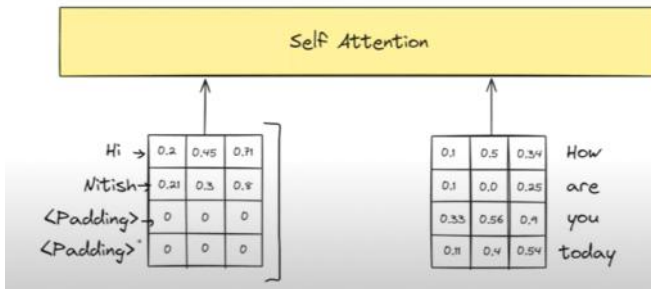
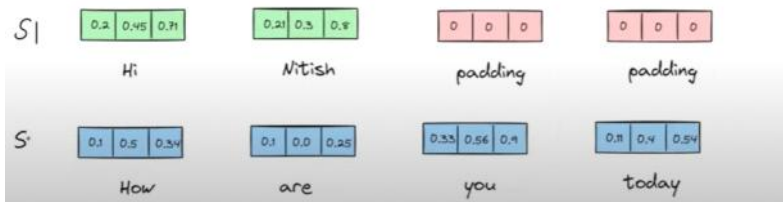
Batch Norm(Revision)

05 June 2024 10:39

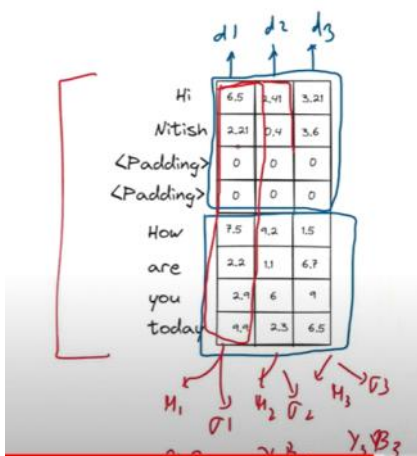


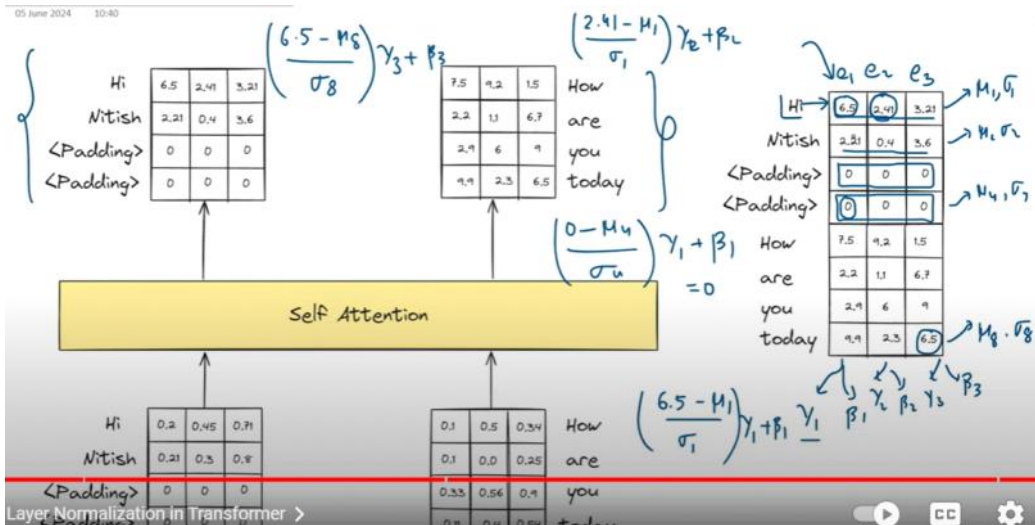
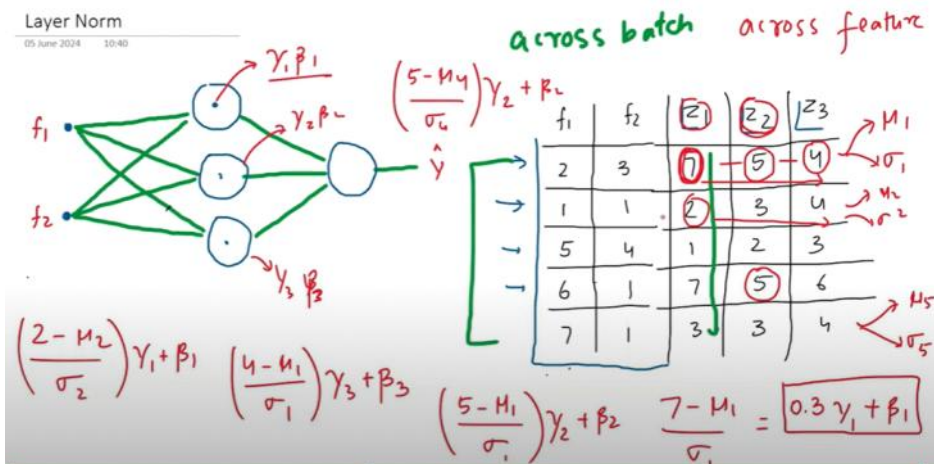
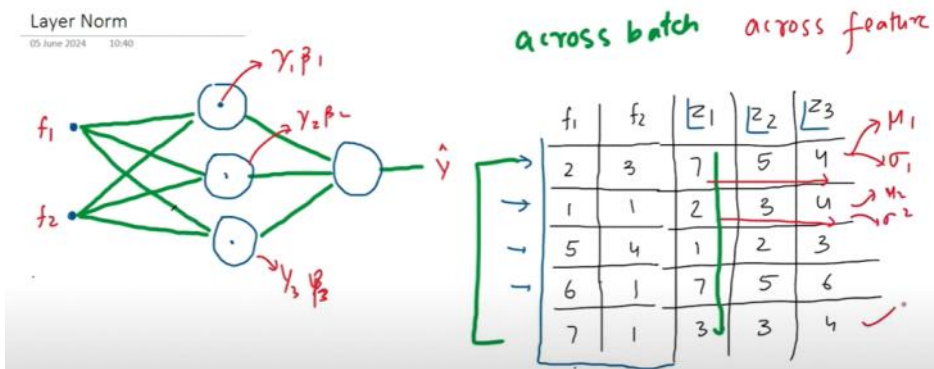
Gamma and Beta are parameters for scaling and shifting, after transforming, it is passed through the activation function and the output is passed to the next hidden layer. Initially gamma = 1 and beta = 0, but these are learnable parameters



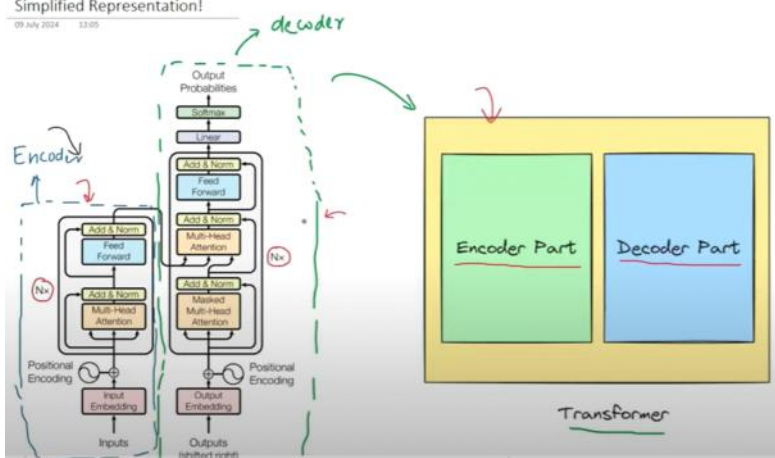


Applying batch normalization as above (Sending 2 sentences at a time as batch): We will get lots a '0' due to padding (unnecessary) when we large number of batch size (i.e. sending more sentences at a time) and having larger sentences. So the mean and STD will not be true representation of the data. Hence in sequential data we can not apply batch normalization. Hence we have to apply layer normalization.

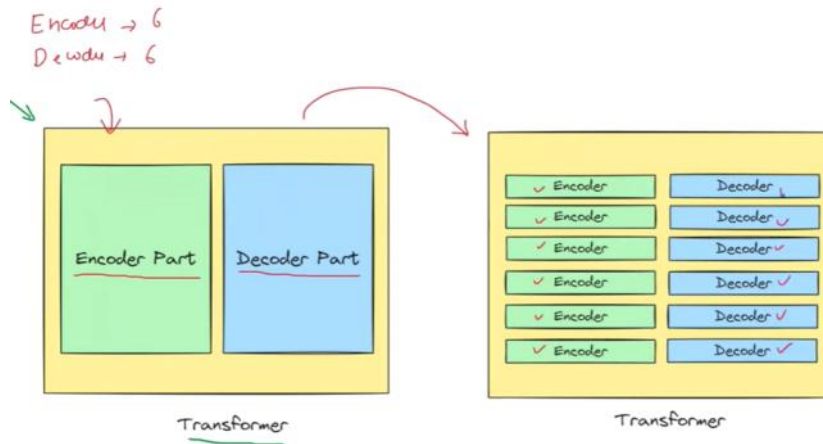




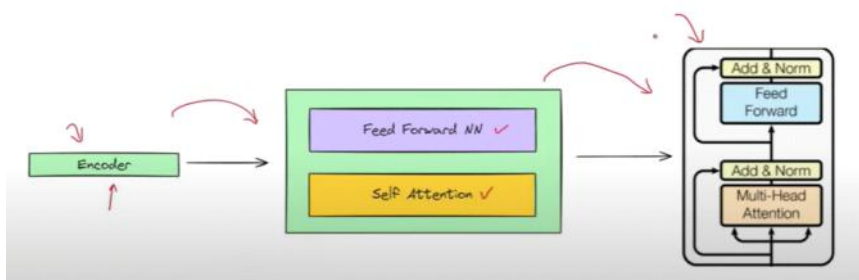
Encoder Architecture:



There are 6 blocks of identical Encoders in the architecture (represents 6 Blocks of architecture). All the encoder blocks are sequentially connected to each other. Input is passed to Encoder 1 and then it moves from encoder 1 to encoder 6 and finally the output is generated which is passed to Decoder. Also decoder has 6 identical Blocks, because 6 blocks gave the best results



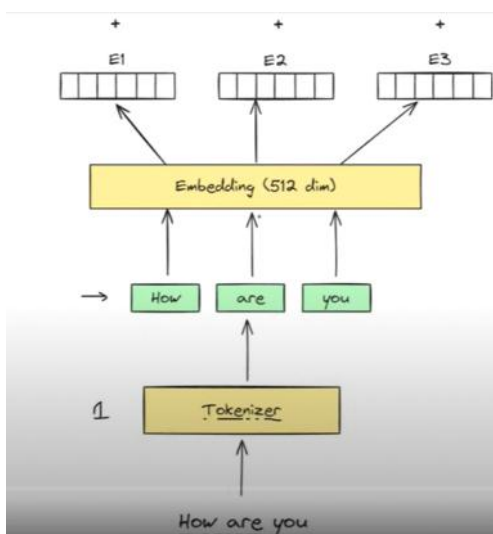
Single Encoder blocks looks as below: It has self attention module and feed forward neural network

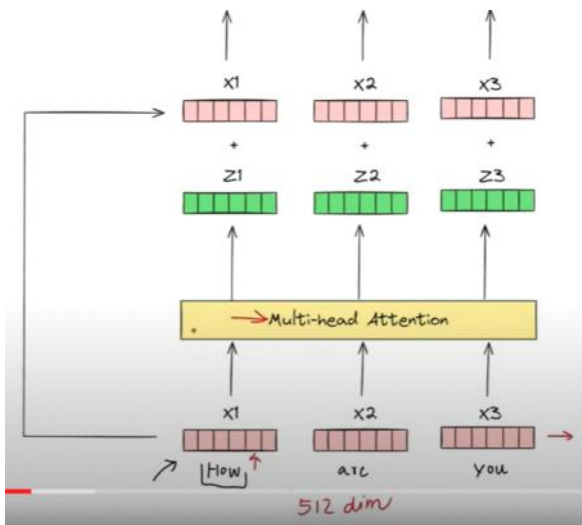
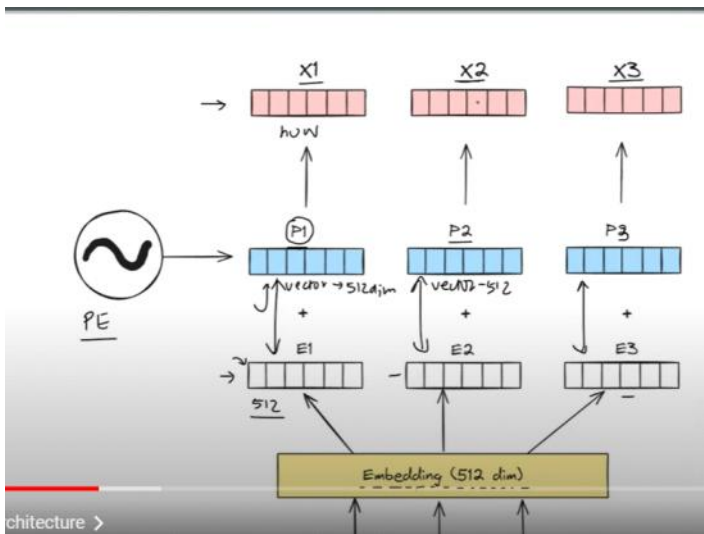


Before the Text/Sentence is passed to the input block, the sentence performs 3 major process as below before it moves in to the input block:

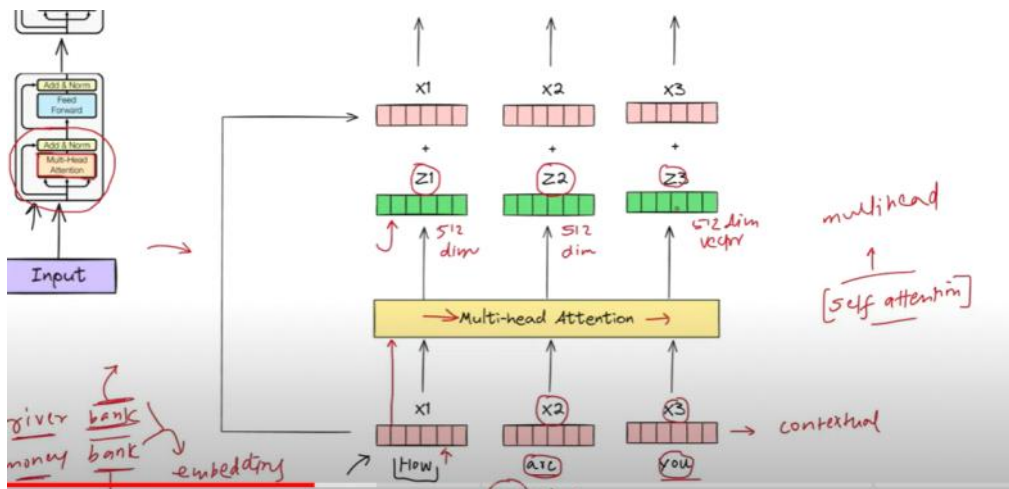
1. Tokenization
2. Text Vectorization (Embedding). i.e. for every word it will create a vector, for every word it will create a 512 dimension vector
3. Positional Encoding and the summation of Word Embedding and Positional Encoding

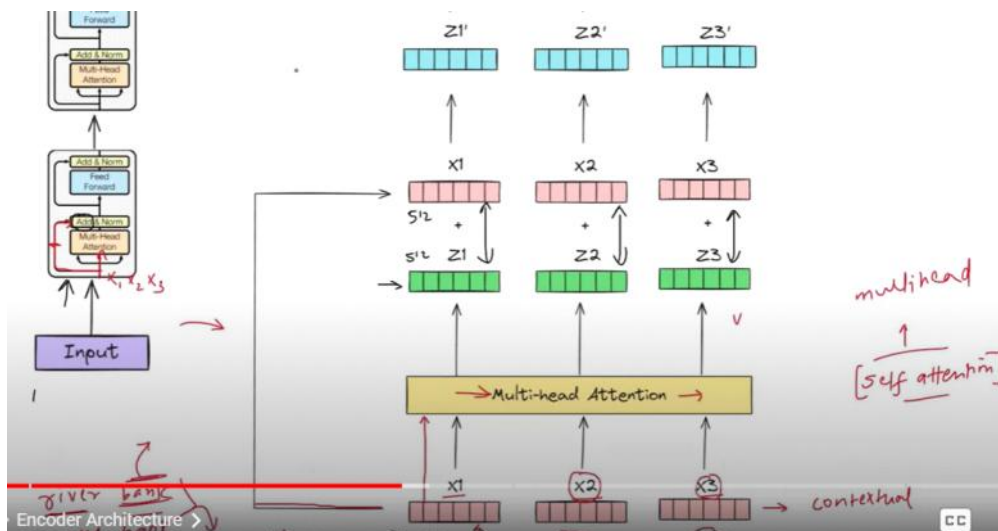
Once these 3 steps are done, the input block is formed. Basically inside the purple input block the above 3 steps are happening.



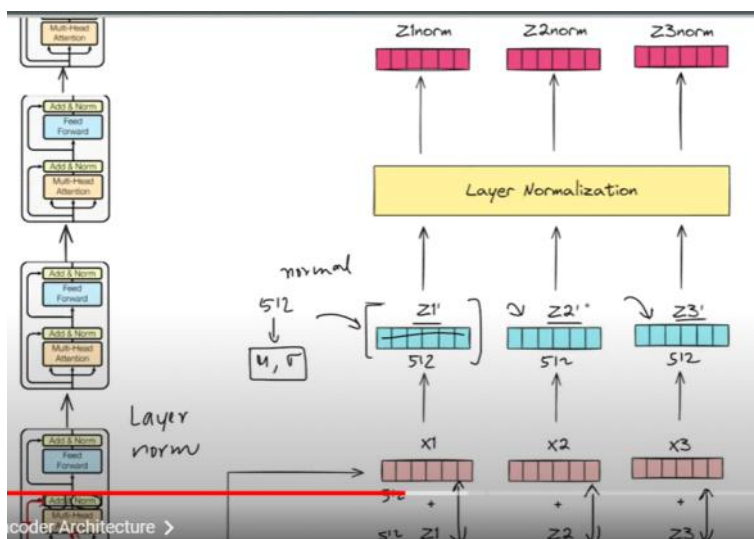


Encoder layer:





The Add & Norm block there is a residual/skip connection, i.e. the original input bypass the multihead attention and then do an addition with the contextual embedding coming out of multihead attention block. Then layer normalization will be performed with the output Z1(dash), Z2(dash), Z3(Dash) and after normalization it will generate Z1norm, Z2norm, Z3norm

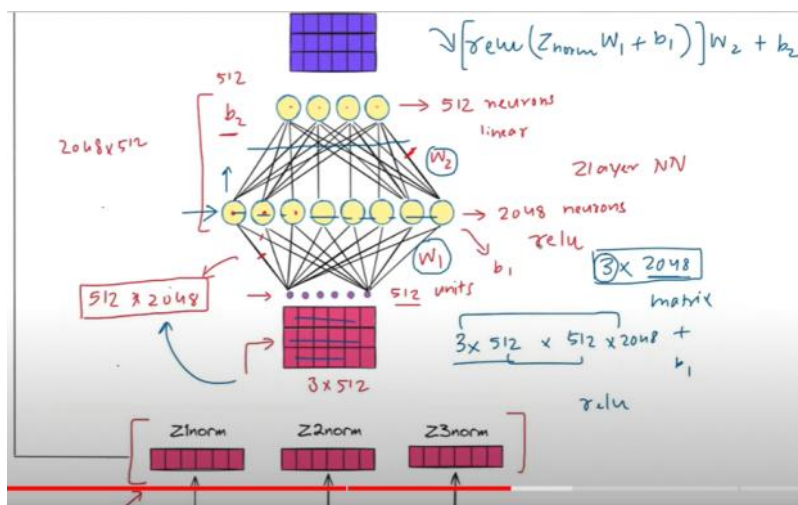


Now the Z1Norm, Z2Norm, Z3Norm will be the input to the feed forward network which is a 2 layer neural network, input layer is of 512 units, the 1st layer is a 2048 neurons with Relu as the activation function. The 2nd layer has 512 neuron and linear is the activation function. All the 3 words together are considered as input for the Feed forward neural network, so it is matrix 3 x 512 .

Now this input 3 x 512 will multiple with 2048 nodes . i.e . $3 \times 512 \times 512 \times 2048$, hence the output will be 3×2048 . This output is passed through Relu and added with Biases. This output is again multiplied with W_2 and added with b_2 .

So, $3 \times 2048 \times 2048 \times 512 = 3 \times 512$

We increased the dimension in the 1st layer and then reduced it back to same dimension in the 2nd layer, so what is the advantage, we were able introduce nonlinearity with the help of Relu.

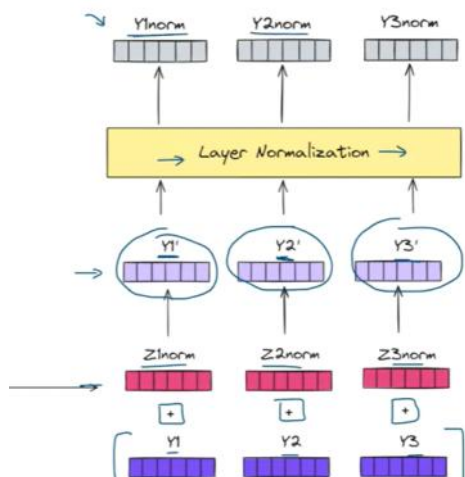
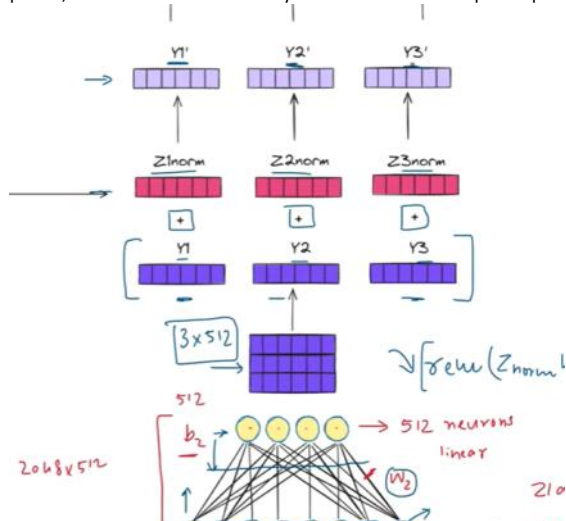


After this it is again Add and Normalization. The reason why it is added is not known. Probably reason might be stable training. The Add & Norm block there is a residual/skip connection, i.e. the original input bypass the Feed Forward Neural network and then do an addition with the output coming out of Neural network block.

Then layer normalization will be performed with the output Y1(dash), Y2(dash), Y3(Dash) and after normalization it will generate Y1norm, Y2norm, Y3norm

Note: Feed Forward Neural network was added to capture the Non-linear complexity in the data.

Note: Why 6 Encoder layers are used? - Since human language is very complex and we need a model with good representation power, and hence it will come only when we have stack up multiple encoder layers.



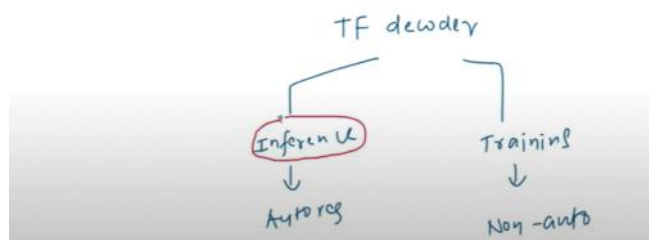
The Y1Norm, Y2Norm, Y3Norm will be passed into the next Encoder block and so on.

Note: The parameters of each encoder blocks (i.e. the weights and Biases of each encoder blocks are different on each of these encoder blocks

Autoregressive models

23 July 2024 17:39

The Transformer decoder is autoregressive at inference time and non-autoregressive at training time.

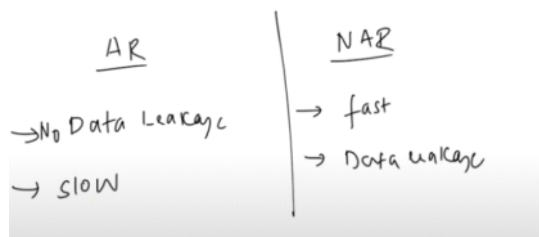


It is behaving differently during prediction and training because of masked self-attention. If the training process is auto-regressive in nature, then the training process will become very slow

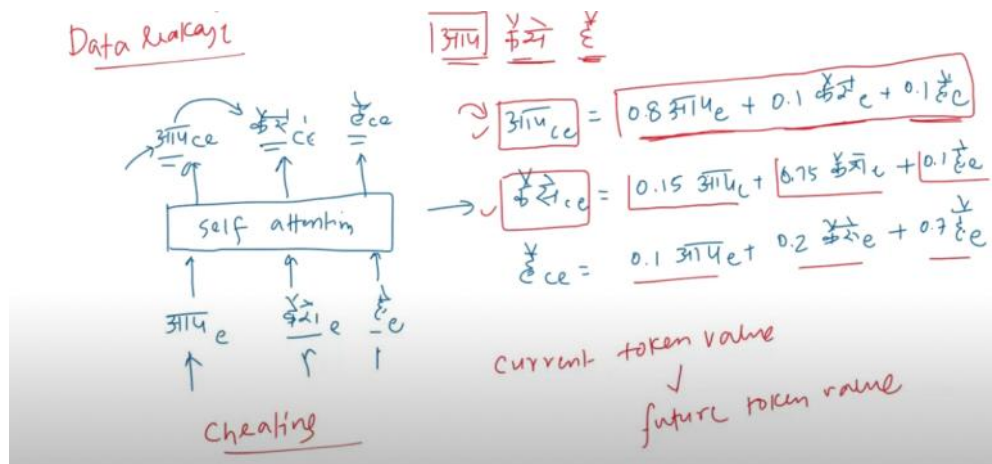
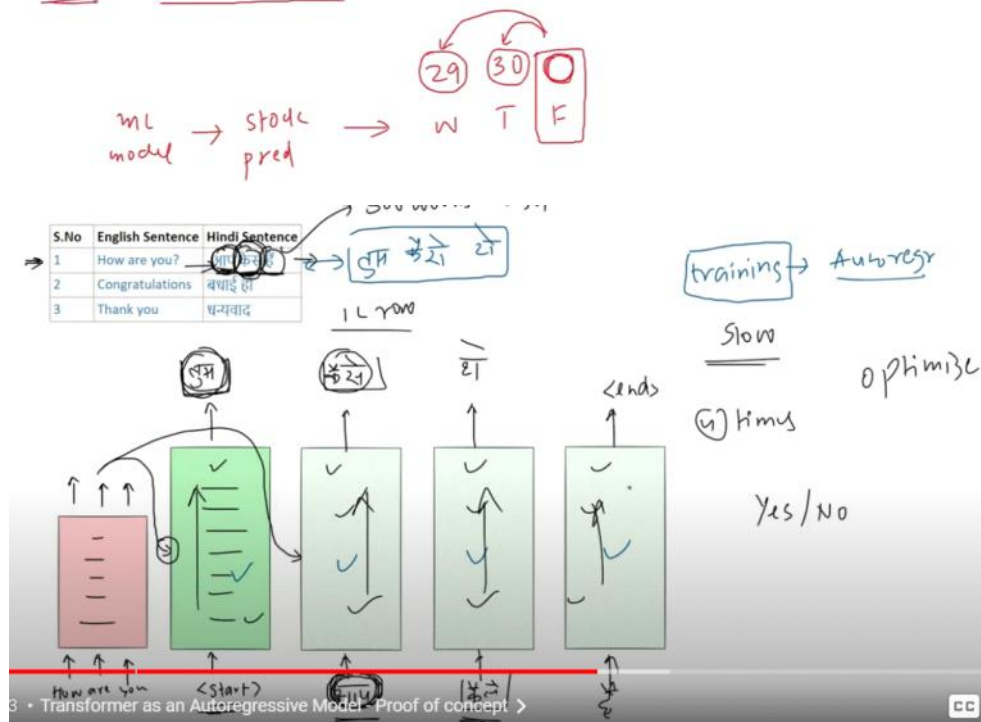
Hence during training we are passing the correct translated word present in the training data (as part of teacher forcing) so the correct translated words can be used to train parallelly and hence during training the decoder can be non-autoregressive in

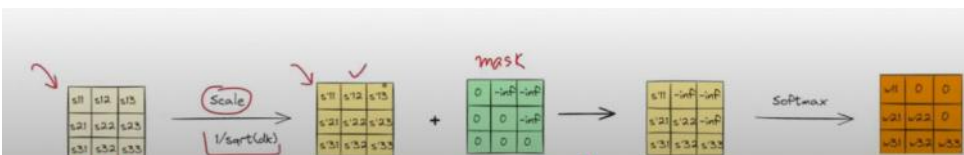
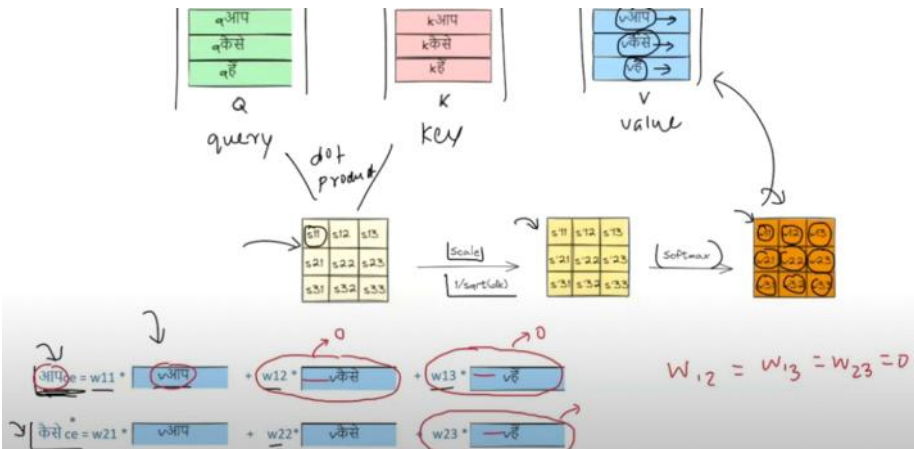
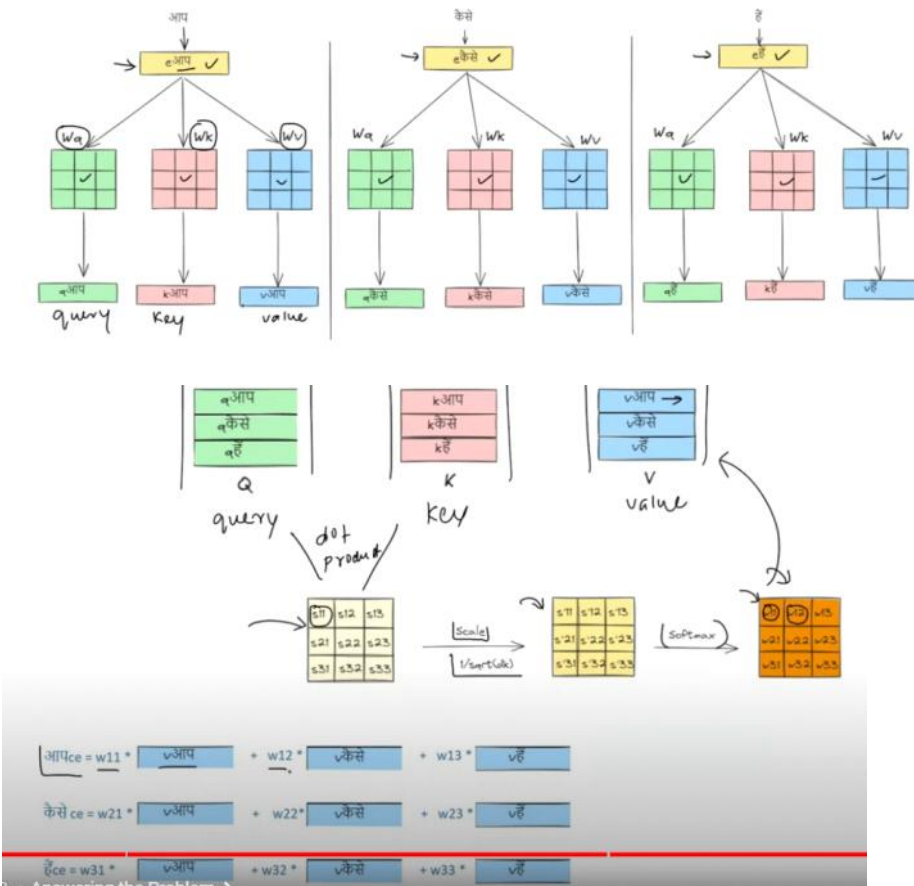
nature. But there is a problem, there is data leakage as contextual embedding of each word with the respect to the future word will be exposed during training, so the model will perform well in training but in prediction phase it will not work well

But during inference/prediction it is by nature it has to auto regressive as previous translated output will act as input for the next timestep



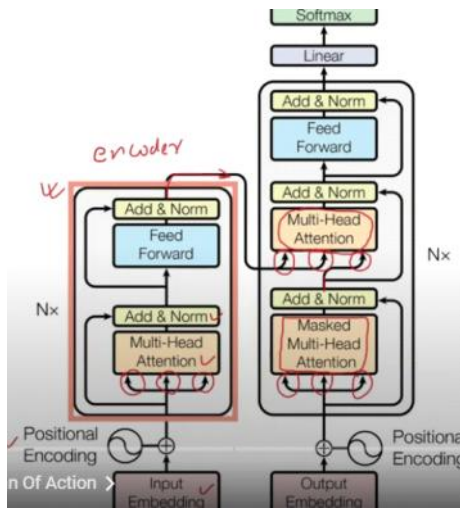
In the context of deep learning, autoregressive models are a class of models that generate data points in a sequence by conditioning each new point on the previously generated points.





So during training phase and parallel processing we do a summation with matrix called mask of same dimension, for the places we want '0' we will have -infinity. The Addition will also provide infinity and then softmax of -infinity is '0'. This is called Masking. So with masking we can do parallel processing and non-auto regressive and no data leakage as we are not showing future words.

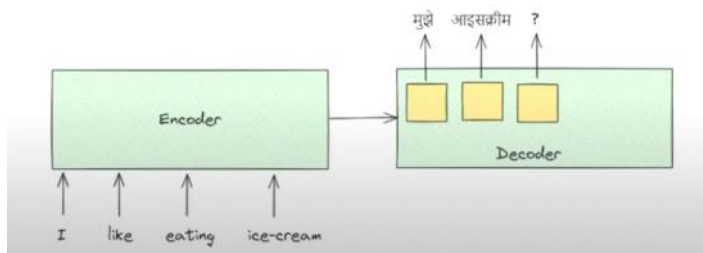
CROSS ATTENTION: The output of the encoder is passed into the multi-head attention module of the decoder



What is Cross Attention

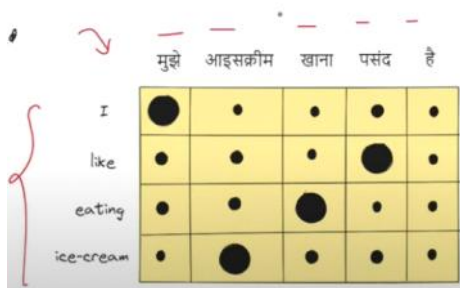
17 August 2024 17:53

Cross-attention is a mechanism used in transformer architectures, particularly in tasks involving sequence-to-sequence data like translation or summarization. It allows a model to focus on different parts of an input sequence when generating an output sequence.



Next word prediction in the decoder is dependent on 2 points:

1. What decoder has generated till now (this is solved by self-attention)
2. What is the input sentence in the encoder (i.e. what information is coming from the encoder) - for this we need to generate a matrix where we will see how the english and Hindi words are related. So to draw the similarity/strong relationship for 2 different sequence (Hindi and English) , it is done by Cross Attention



Cross Attention is conceptually very similar to Self-Attention

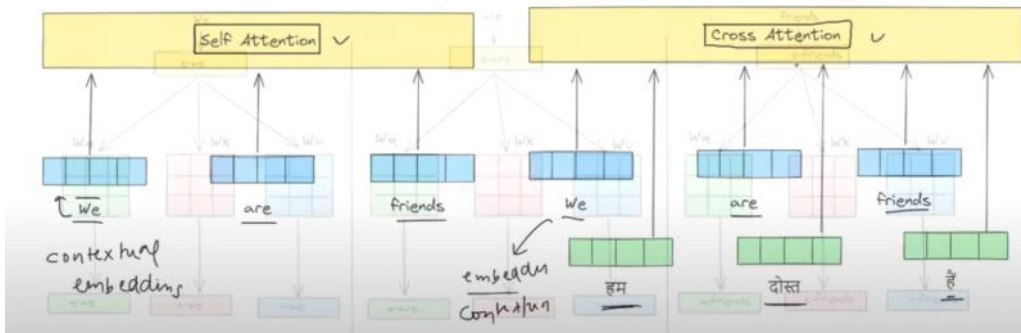
Self-Attention Vs Cross Attention

1. The input ✓
2. The processing ✓
3. The output ✓

Difference b/w self attention and Cross attention:

Self-Attention Vs Cross Attention (Input)

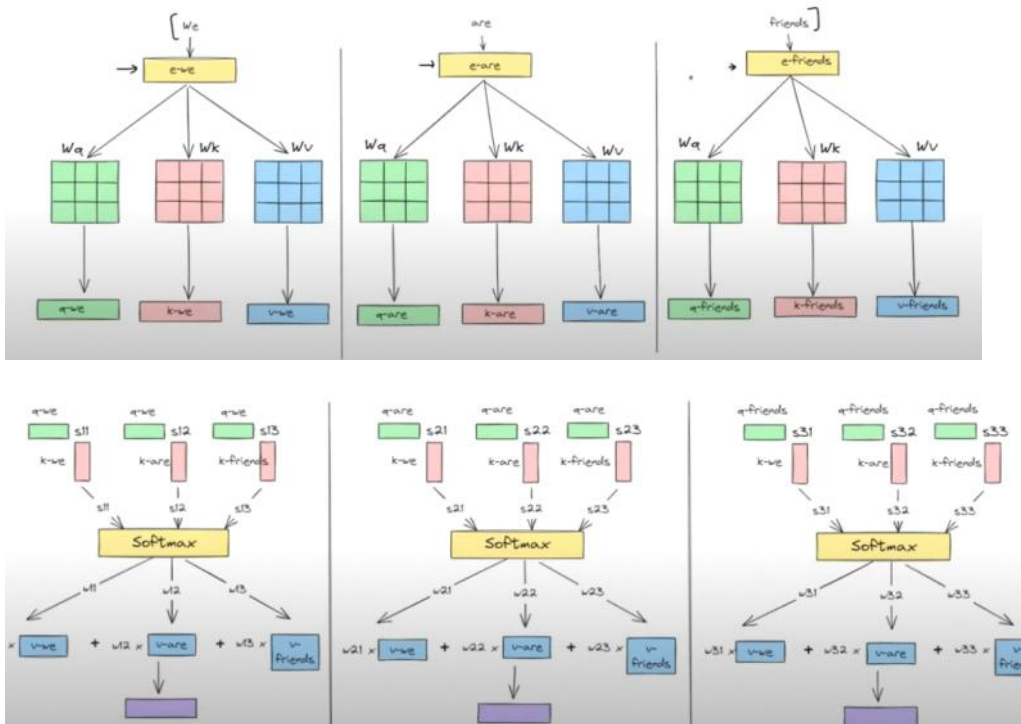
13 August 2024 08:22



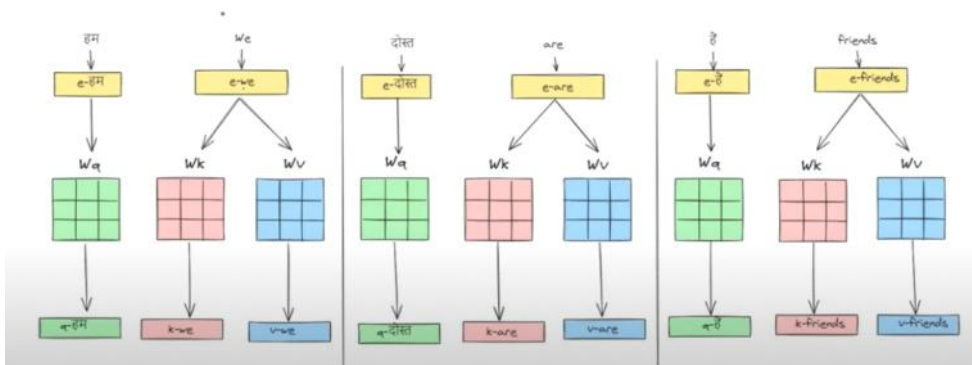
Self attention Processing as below:

Self-Attention Vs Cross Attention (Processing)

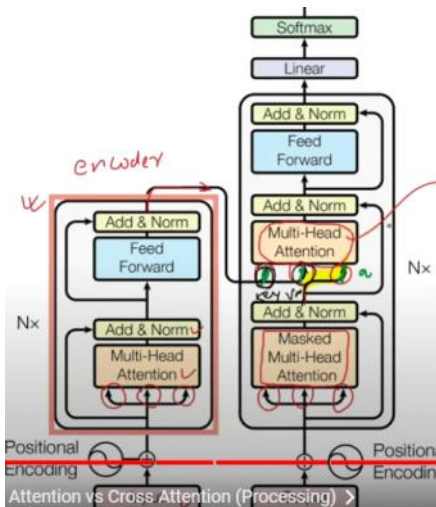
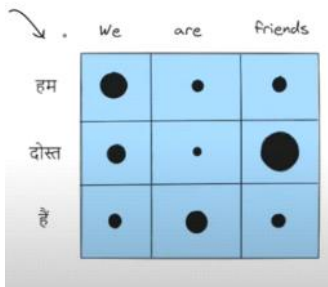
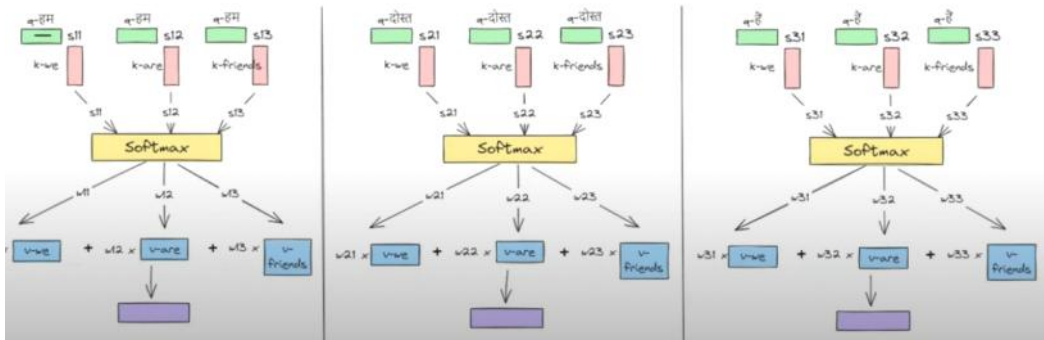
12 August 2024 17:54



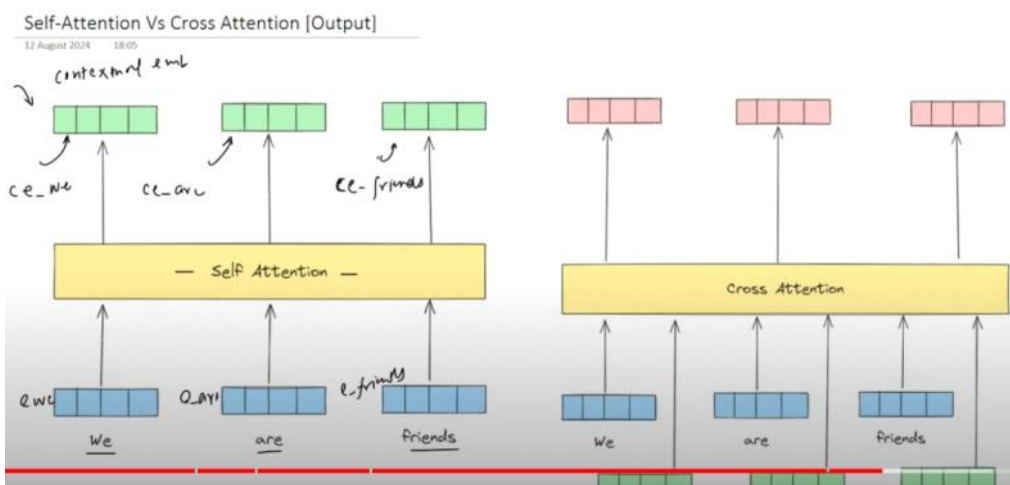
Cross attention Processing as below:



In cross-attention, the query vector is coming from the output sequence. While Key and value vector is coming from input sequence (i.e. Encoder embedding)



Output difference b/w self attention and cross attention as below:



Contextual Embedding is shown as below:

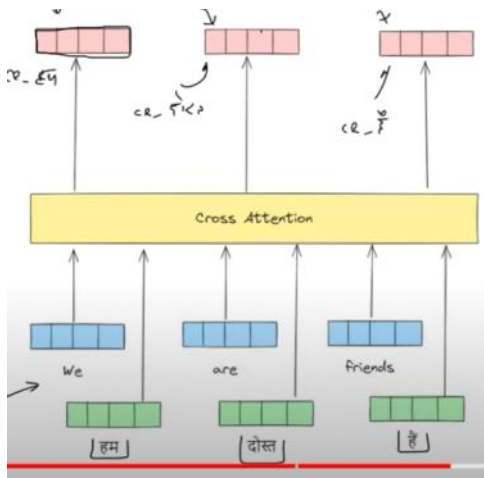
$$ce_we = 0.8 \times e_we + 0.1 \times e_are + 0.1 \times e_friends$$

$$ce_are = 0.15 \times e_we + 0.75 \times e_are + 0.1 \times e_friends$$

$$ce_friends = 0.2 \times e_we + 0.1 \times e_are + 0.7 \times e_friends$$

Cross attention contextual embedding will provide contextual embedding for the Hindi words (i.e. output words)

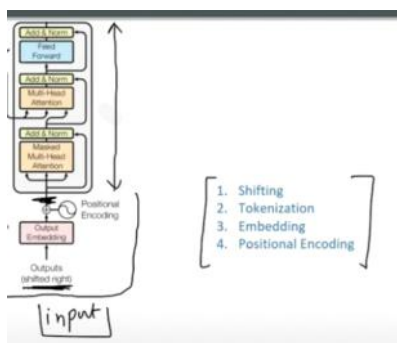
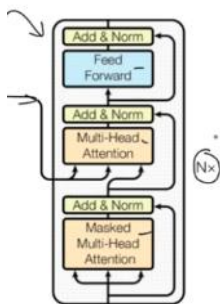
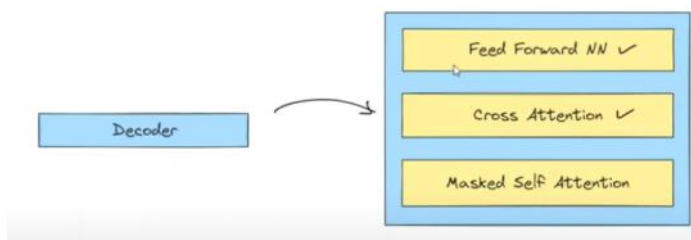


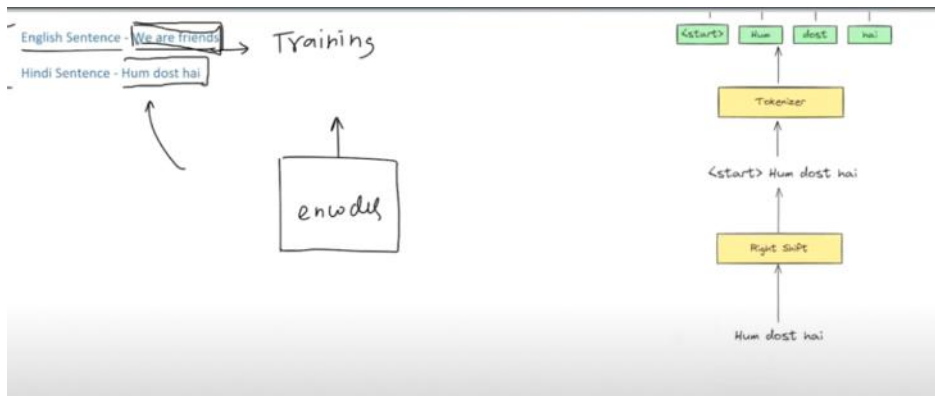


$$\left\{ \begin{array}{l} \text{ce_हम} = 0.5 \times e_we + 0.3 \times e_are + 0.2 \times e_friends \\ \text{ce_दोस्त} = 0.2 \times e_we + 0.2 \times e_are + 0.6 \times e_friends \\ \text{ce_हैं} = 0.3 \times e_we + 0.4 \times e_are + 0.3 \times e_friends \end{array} \right.$$

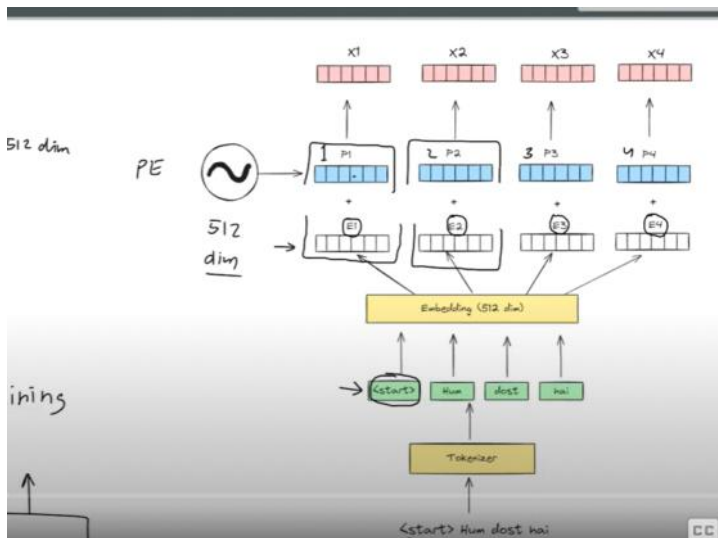
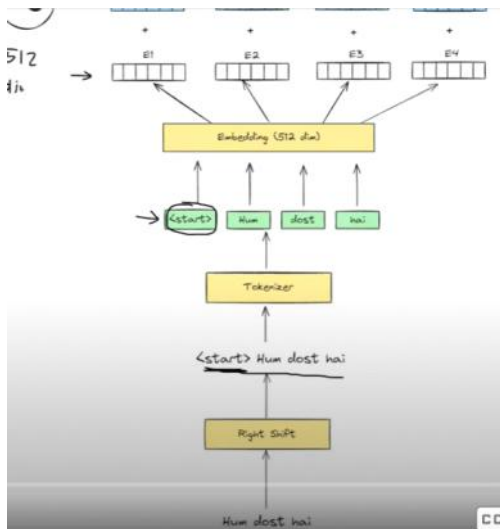
Use cases of Cross attention:

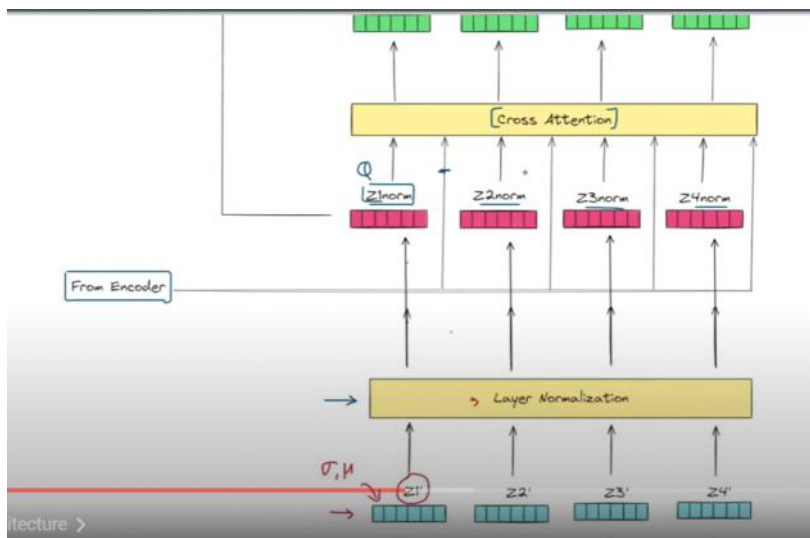
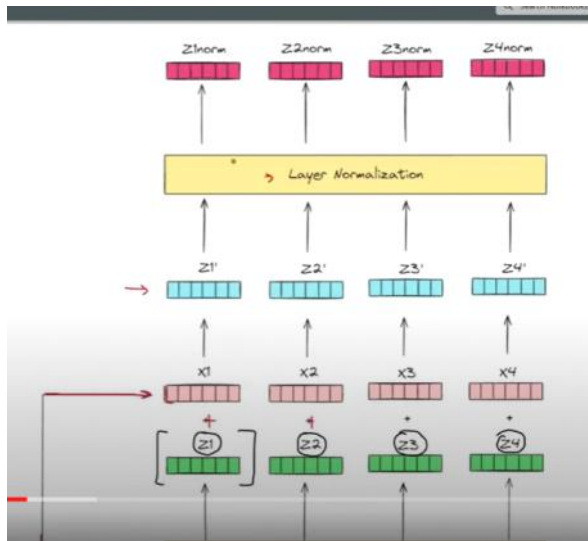
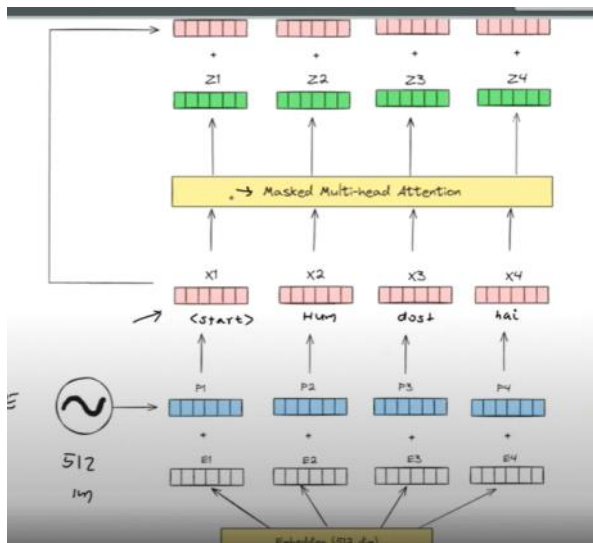
1. Language translation
2. Multimodal application: e.g. text to speech, text to image conversion

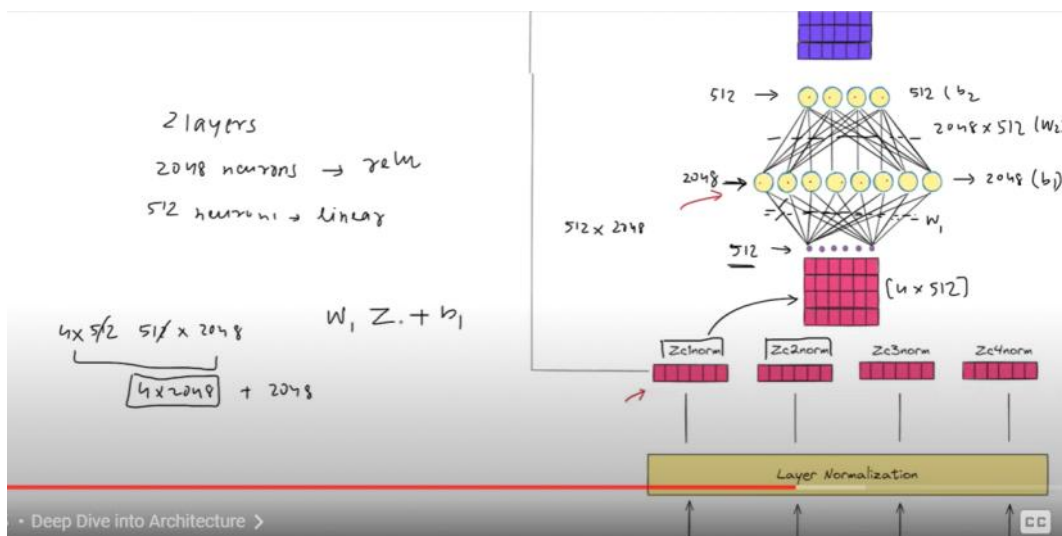
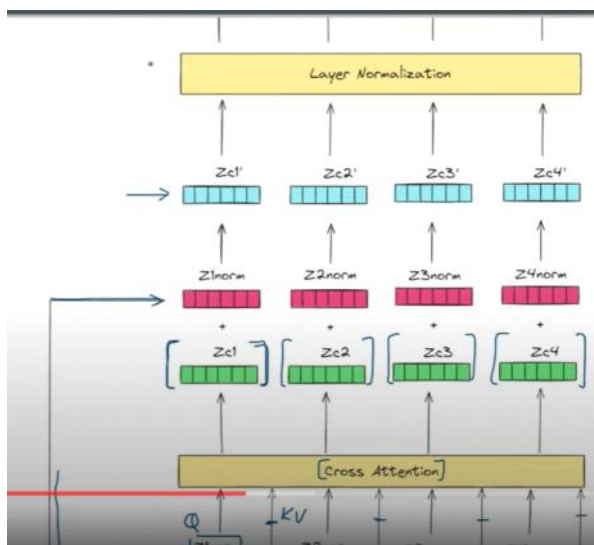
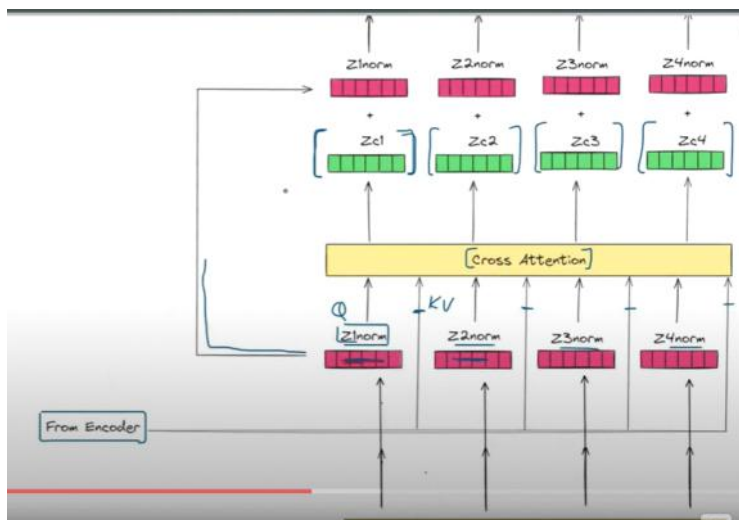


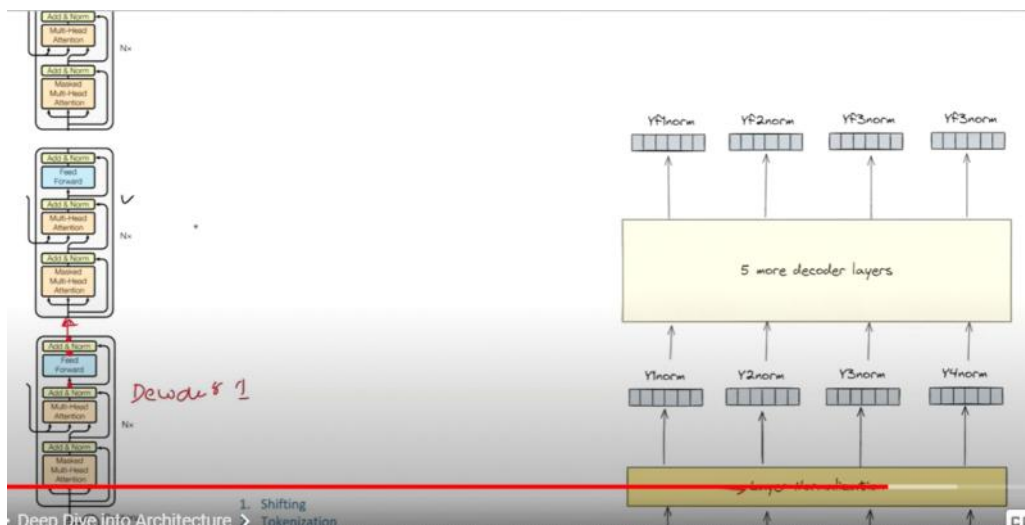
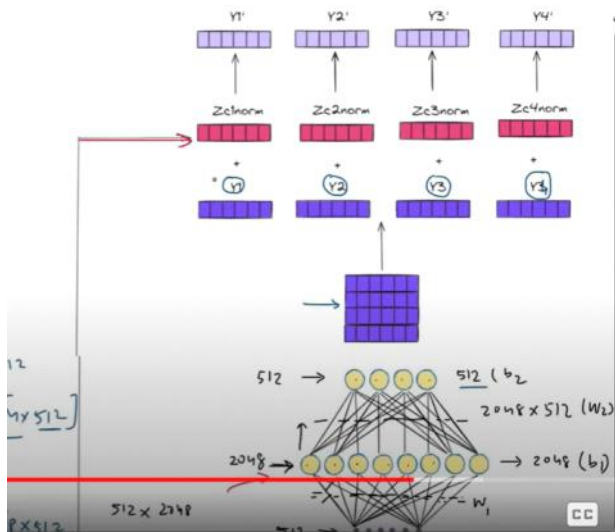


Right Shifting Step: will add <START> token and shift the sequence of words



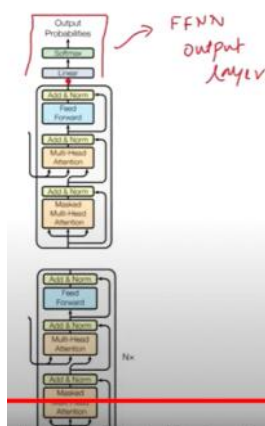






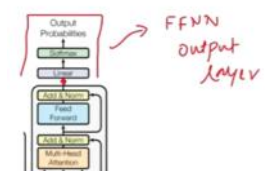
Decoder Architecture

22 August 2024 00:56



Decoder Architecture

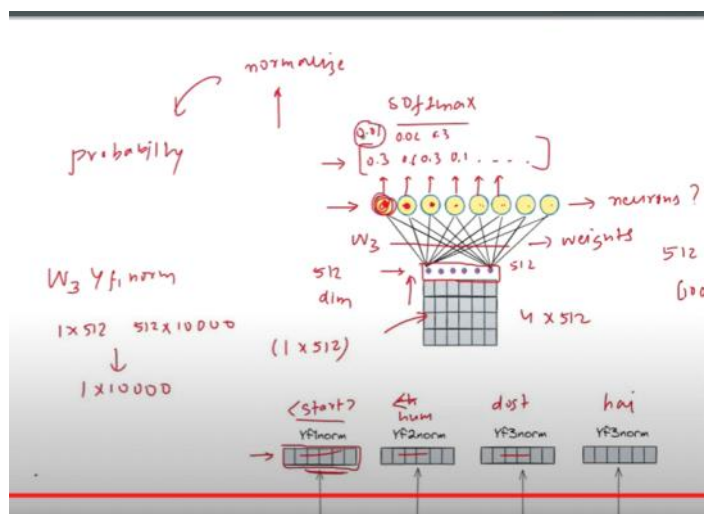
22 August 2024 00:56



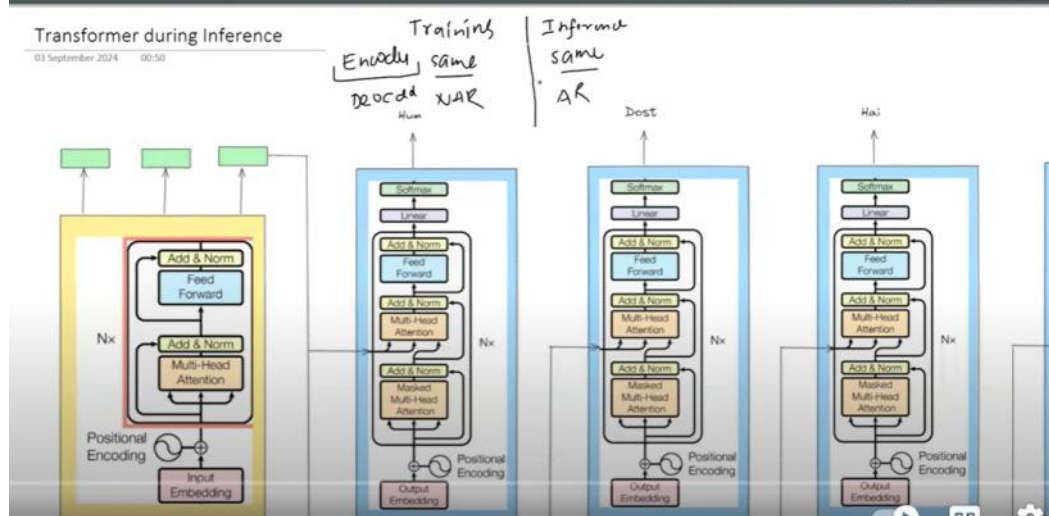
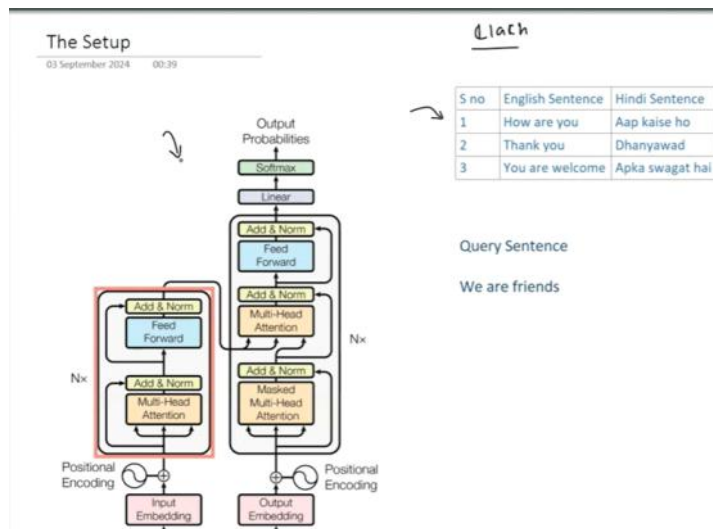
The linear layer is nothing but a Feed Forward Neural network with the number of neurons (called as 'V') is equal to number of unique vocabulary words present in the translated training sentence (i.e. Hindi) . These neurons are also called as Logits. Each neuron will represent each of the Hindi words.

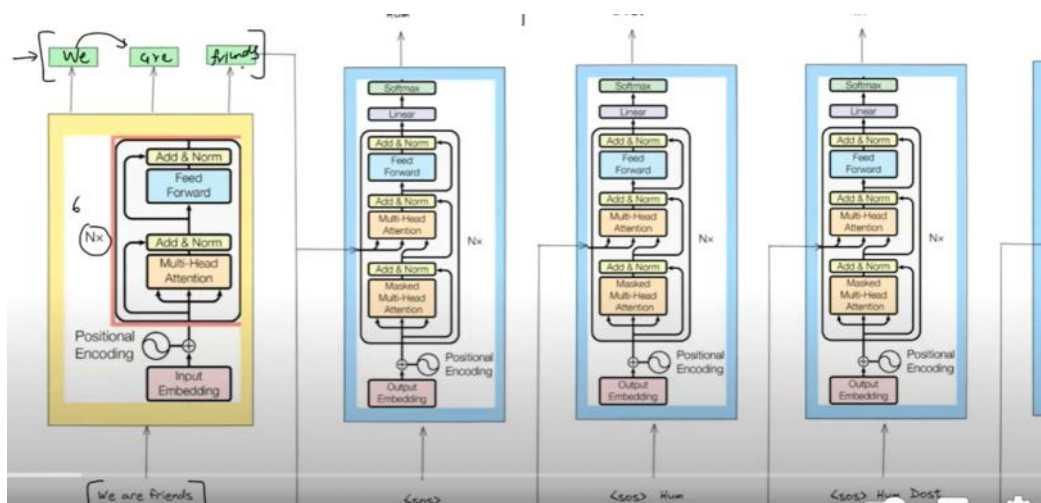
The output of this Neural network will be passed through a softmax layer which will provide a probability value for each of the Hindi Words. Similarly the 4 normalized output of the below decoder layer are sent parallelly to linear Feed forward network and

once it receives STOP word, the output probability will be stopped.

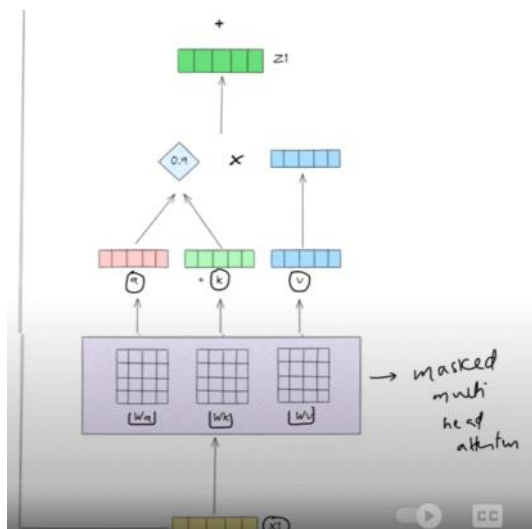
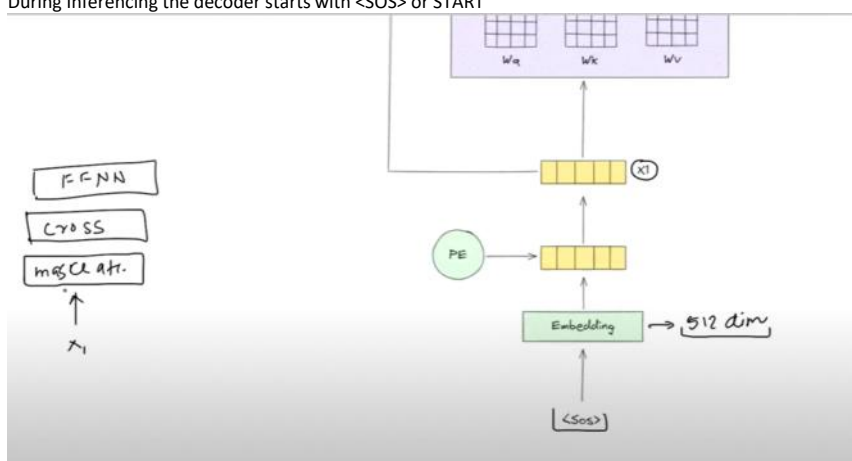


DEODER WORKING DURING INFERENCE/PREDICTION:

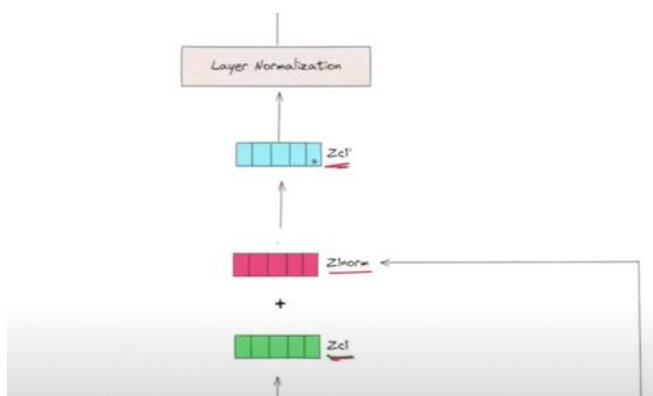
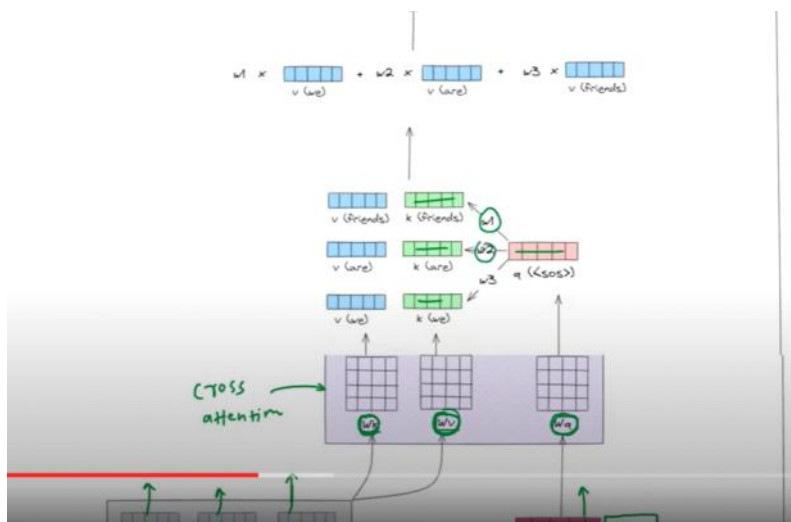
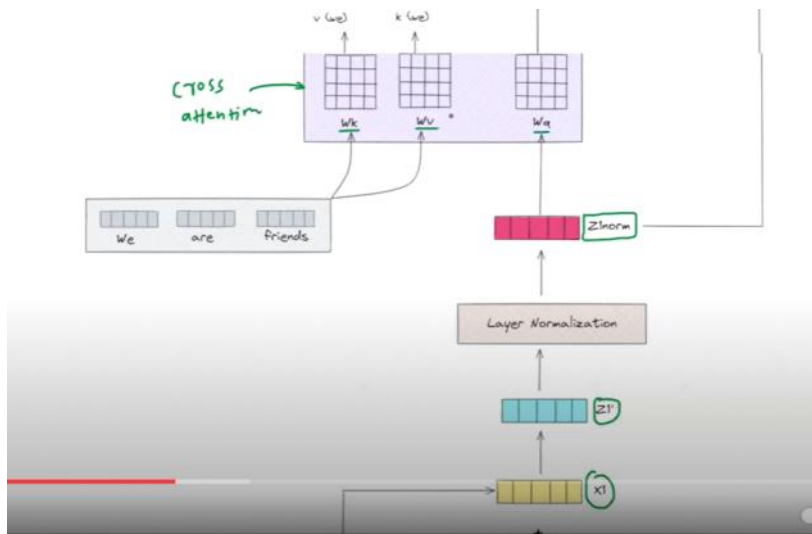
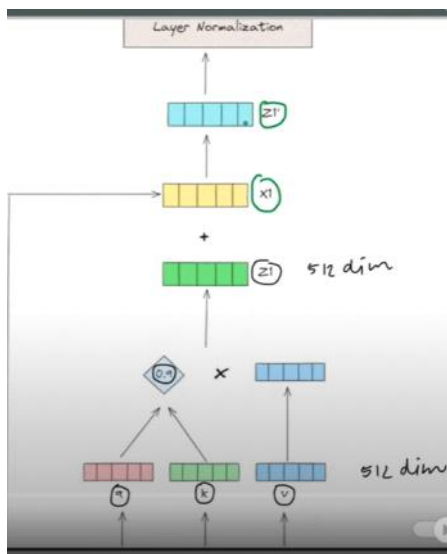


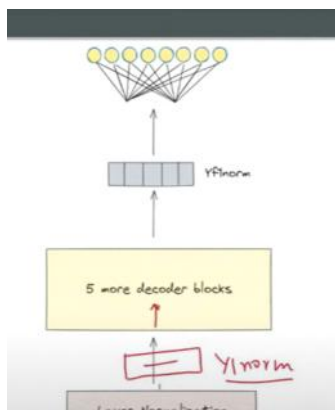
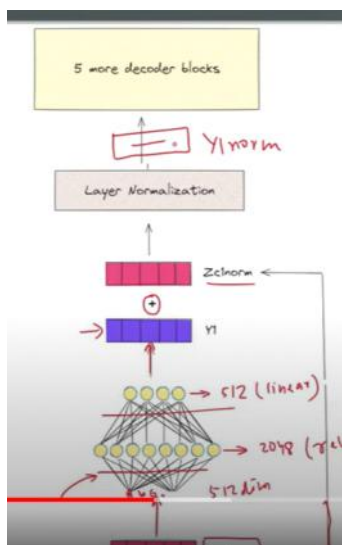
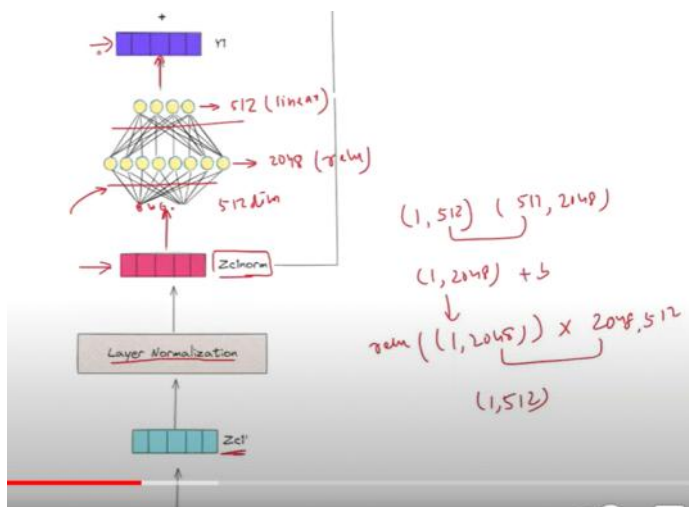
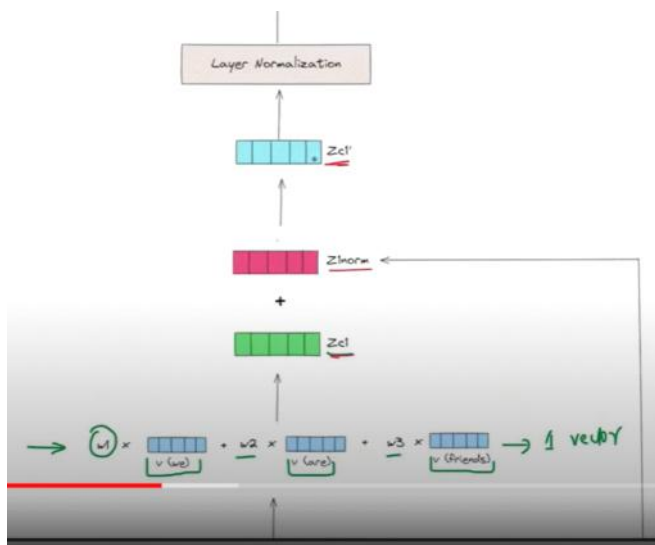


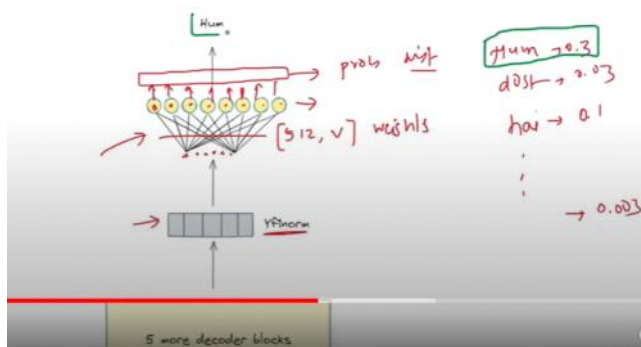
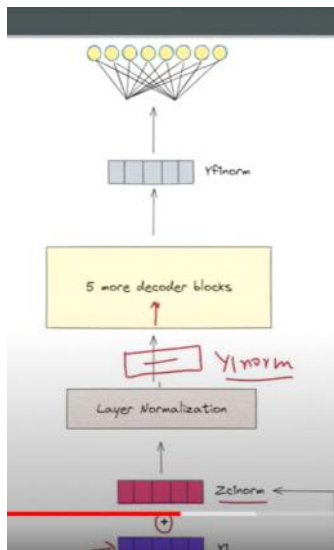
During Inferencing the decoder starts with <SOS> or START



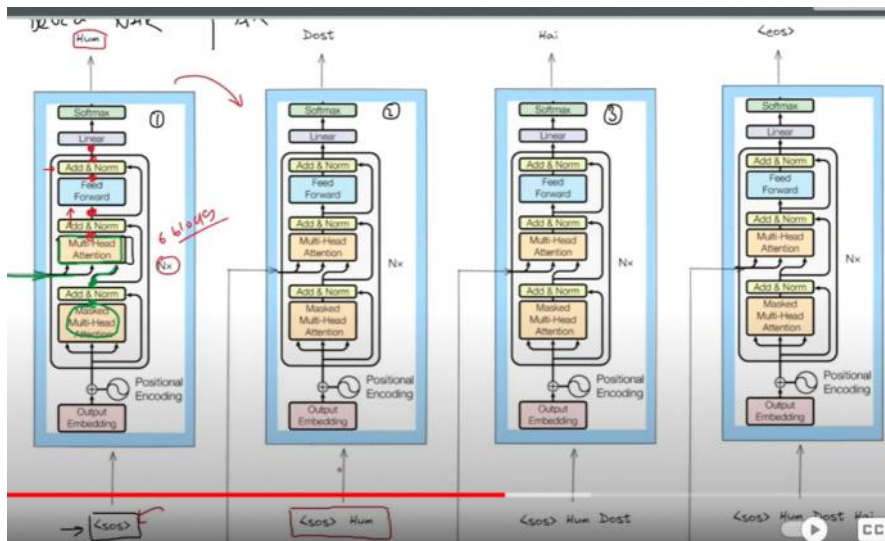
0.9 (e.g.) is the attention score, i.e. $W1$

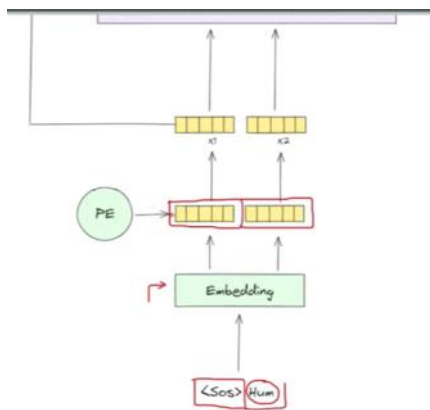




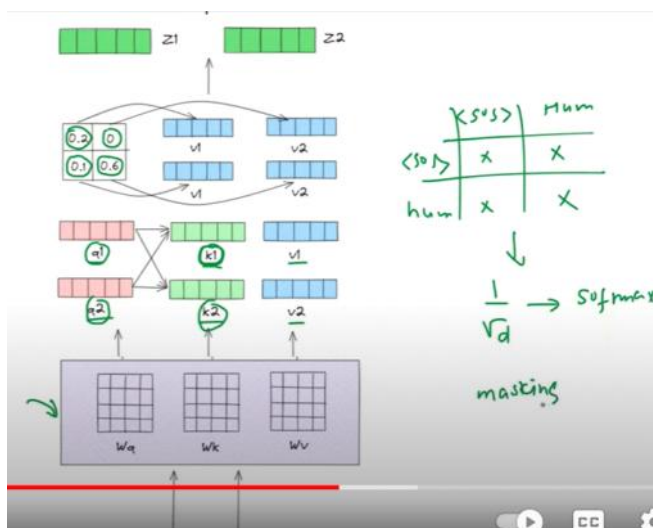
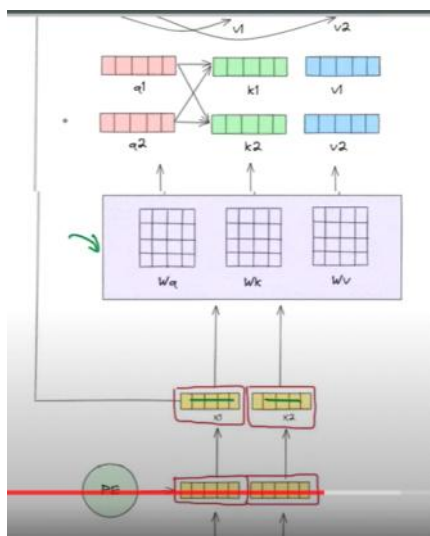


Now when the Hindi word "Hum" have been generated in the 1st timestep during the inference phase of the decoder where the input was <SOS>, in the 2nd timestep the input of the decoder (during the inference phase) will be <SOS> and <HUM>. Similarly for further timestep it will keep on added.





Since we have 2 tokens (SOS, HUM) hence we are getting X1 and X2

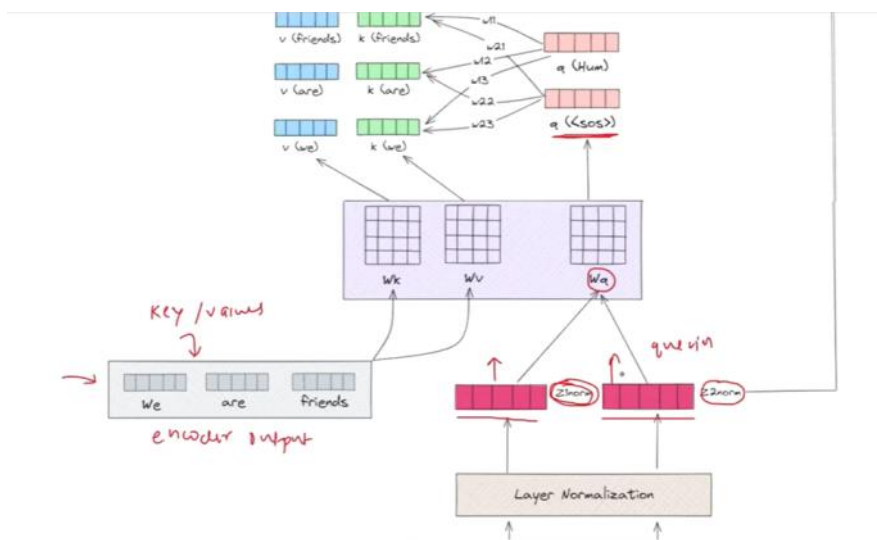
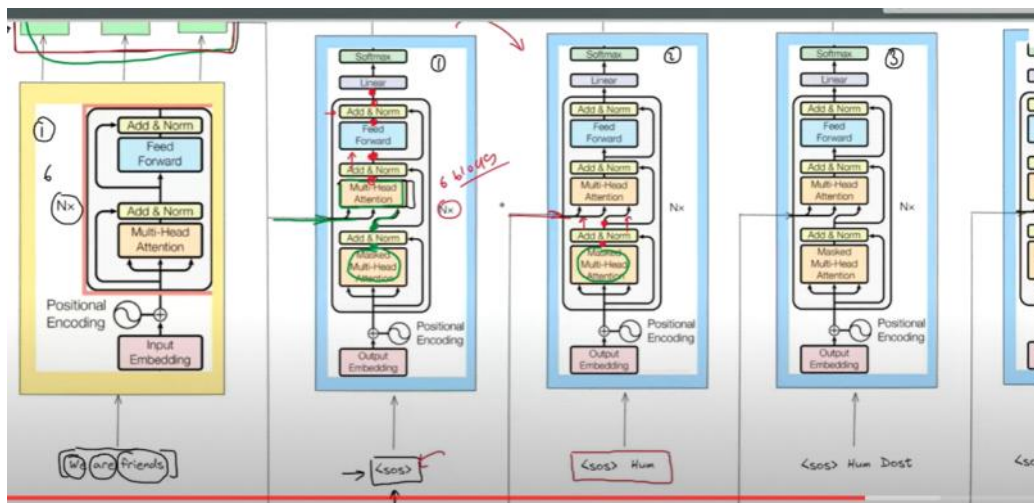
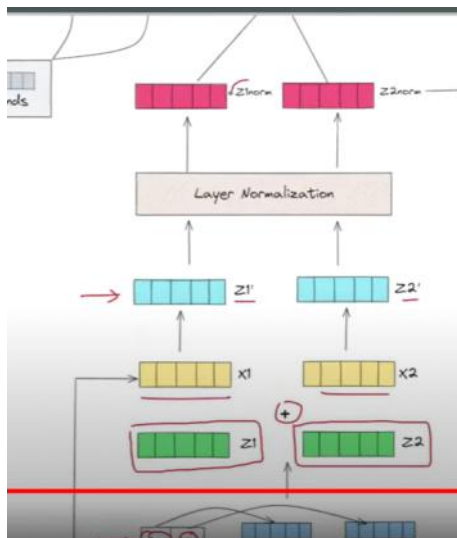


Masking: <SOS> should not see <HUM> but <HUM> can see <SOS>, so the dot product matrix of <SOS> and <HUM> is '0' (upper right corner).

Note: Masking happens both during training and inferencing. Since Masking was done during training, masking has to be done during inferencing, if we do not do it, then there will be a shift and quality of prediction will not be good.

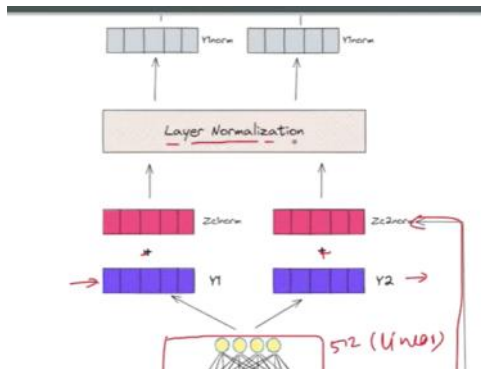
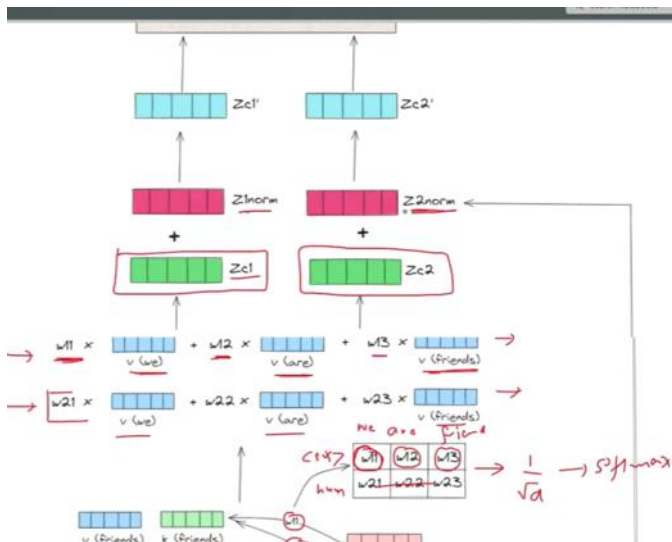
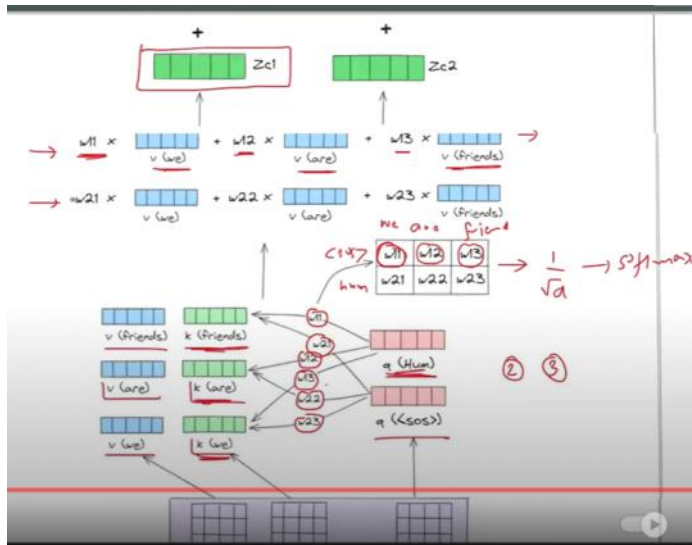
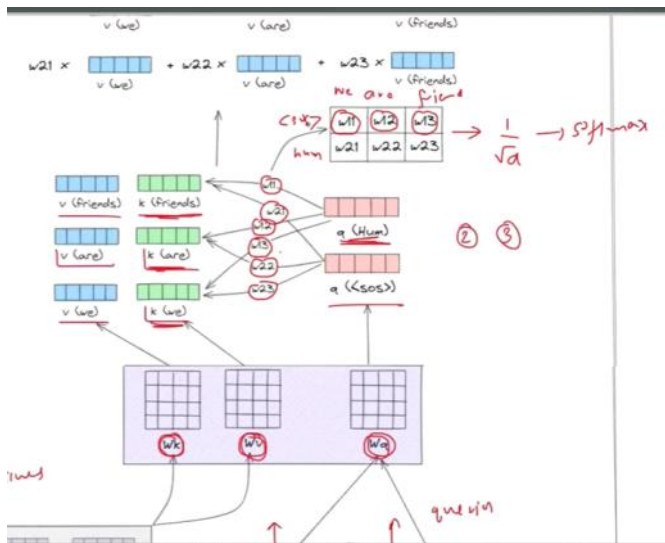
	sos	hum	dostr
sos	x	0	0
hum	x	x	0
dostr	x	x	x

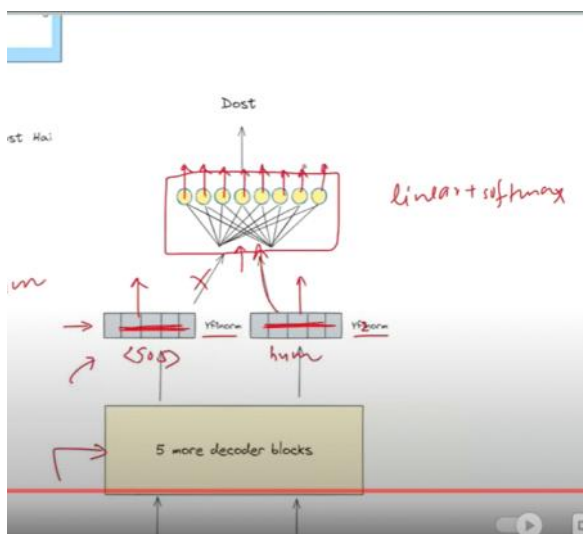
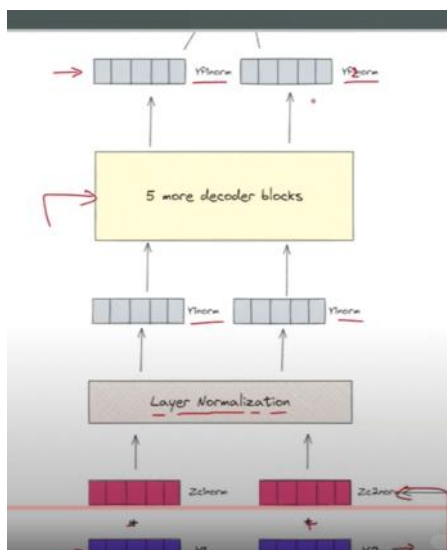
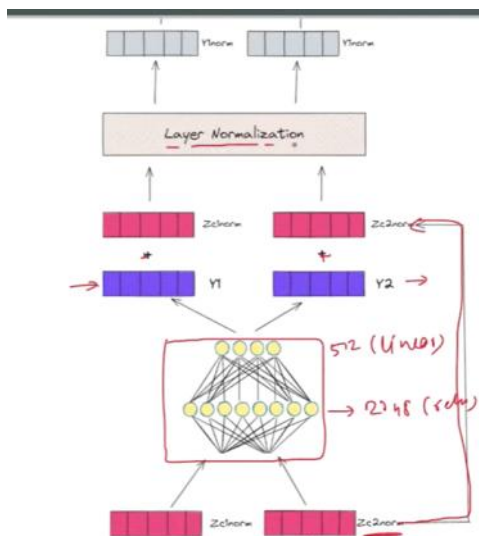
	<u>sos</u>	<u>hum</u>	<u>dost</u>
<u>sos</u>	X	0	0
<u>hum</u>	X	X	0
<u>dost</u>	X	X	X



$$w_{21} \times \begin{bmatrix} v(w_e) \\ v(w_e) \end{bmatrix} + w_{22} \times \begin{bmatrix} v(w_e) \\ v(w_e) \end{bmatrix} + w_{23} \times \begin{bmatrix} v(w_e) \\ v(w_e) \end{bmatrix}$$

$$\begin{bmatrix} w_{21} & w_{22} & w_{23} \end{bmatrix} \begin{bmatrix} v(w_e) \\ v(w_e) \\ v(w_e) \end{bmatrix} \rightarrow \frac{1}{\sum} \rightarrow \text{softmax}$$





Since inferencing is autoregressive in nature, so only 1 word, the previous generated word will be passed to the linear Neural network and softmax. In this case, <HUM> is passed to the Neural network and softmax for the final prediction of next word